GarageGo

Documentación técnica del proyecto

Aplicación móvil y web para reservar y pagar cocheras particulares por hora, con panel de administración

Producto	GarageGo (app Android / iOS / web + panel de administración web)
Versión documentada	1.0.0 · rama `feature/ultimas-funcionalidades` (último commit 361d411 del 16/10/2026, con las migraciones 0001 a 0082; la 0083 se suma en el próximo commit)
Fecha	Octubre de 2026
Equipo	Bruno Bellot · Nicolás Porcel · Ignacio Quintana · Braian Carballo · Gianluca Polizzi
Institución · materia · docente	Instituto Tecnológico Beltrán · Tecnicatura en Análisis de Sistemas · Prácticas Profesionalizantes III · Docente: Silvio Daniel Benítez
Repositorio	github.com/porcelcabralnicolas-27/GARAGEGO-2027-CLAUDE
Versión web publicada	https://garagego.garagego2027.workers.dev
Panel de administración	https://garagego-admin.garagego2027.workers.dev

Índice

Acerca de este documento 5
1. Presentación general del proyecto 6
1.1 Nombre del producto 6
1.2 Integrantes y roles del equipo 6
1.3 Problema o necesidad identificada 6
1.4 Objetivo general del sistema 6
1.5 Público o tipo de usuario 7
1.6 Alcance del producto 7
1.7 Funcionalidades incluidas y fuera del alcance 7
2. Requerimientos 9
2.1 Requerimientos funcionales 9
2.2 Requerimientos no funcionales 11
2.3 Reglas de negocio principales 12
2.4 Restricciones conocidas del proyecto 13
3. Usuarios, actores y casos de uso 15
3.1 Tipos de usuarios y actores 15
3.2 Permisos y responsabilidades 15
3.3 Diagrama general de casos de uso 16
3.4 Descripción de los casos de uso principales 18
4. Historias de usuario y planificación 24
4.1 Historias de usuario principales 24
4.2 Criterios de aceptación 25
4.3 Backlog del producto 27
4.4 Priorización de funcionalidades 28
4.5 Registro de tareas realizadas, pendientes y descartadas 28
4.6 Evolución del proyecto durante la cursada 29
5. Diseño y arquitectura de la solución 31
5.1 Diagrama general de arquitectura 31
5.2 Frontend, backend, base de datos, APIs y servicios externos 33
5.3 Comunicación entre los componentes 33
5.4 Decisiones de diseño relevantes 34
5.5 Patrones y criterios de arquitectura 35
5.6 Organización del código del cliente 36
6. Tecnologías utilizadas 37
6.1 Lenguajes 37

6.2 Frameworks y bibliotecas de la app 37
6.3 Panel de administración 38
6.4 Backend y base de datos 39
6.5 Servicios externos 39
6.6 Herramientas de desarrollo, pruebas y control de versiones 39
6.7 Despliegue y distribución 40
7. Modelo de datos 42
7.1 Diagrama entidad-relación 42
7.2 Principales entidades 44
7.3 Relaciones entre los datos 45
7.4 Restricciones y validaciones importantes 45
7.5 Criterios para proteger la integridad de la información 46
7.6 Ciclo de vida de una reserva 46
7.7 Funciones principales de la base 48
8. Diseño de interfaces 49
8.1 Principales pantallas del sistema 49
8.2 Flujo de navegación 53
8.3 Criterios de experiencia de usuario 55
8.4 Validaciones de los formularios 55
8.5 Tratamiento de errores y mensajes al usuario 56
8.6 Accesibilidad 56
9. Seguridad 58
9.1 Registro, autenticación y autorización 58
9.2 Roles y permisos 58
9.3 Protección de contraseñas y datos sensibles 59
9.4 Validación de las entradas 59
9.5 Manejo de sesiones y tokens 59
9.6 Riesgos identificados y controles implementados 60
9.7 Aspectos de seguridad pendientes 60
10. Pruebas realizadas 62
10.1 Estrategia de prueba 62
10.2 Casos de prueba principales 62
10.3 Pruebas funcionales realizadas 64
10.4 Errores detectados y forma de resolución 65
10.5 Limitaciones y funcionalidades que todavía requieren pruebas 66
11. Instalación y despliegue 67

11.1 Requisitos previos 67
11.2 Instrucciones para instalar y ejecutar 67
11.3 Variables de entorno necesarias 67
11.4 Configuración de la base de datos 68
11.5 Datos y usuarios iniciales para pruebas 68
11.6 Procedimiento de despliegue 69
11.7 Dirección de acceso 69
12. Repositorio y control de versiones 70
12.1 Enlace al repositorio 70
12.2 Archivo README 70
12.3 Organización de carpetas y componentes 70
12.4 Instrucciones de ejecución 70
12.5 Historial de commits 71
12.6 Versión presentada 71
13. Estado final y conclusiones 72
13.1 Funcionalidades terminadas 72
13.2 Funcionalidades parcialmente desarrolladas 72
13.3 Problemas conocidos 72
13.4 Decisiones que debieron modificarse 72
13.5 Principales dificultades encontradas 73
13.6 Logros del equipo 74
13.7 Aprendizajes obtenidos 74
13.8 Mejoras y próximos pasos 74
Anexo A. Sugerencias: qué falta para cumplir la consigna 75
Anexo B. Glosario 76
Anexo C. Migraciones por etapa 77

Acerca de este documento

Este documento presenta **cómo se construyó GarageGo y qué se aprendió en el proceso**. Sigue, sección por sección, la estructura pedida en la consigna de la presentación técnica y la completa con anexos. El manual de usuario se entrega como un documento aparte. La idea es la de una *software factory*: que otra persona pueda entender el producto, usarlo, mantenerlo y continuarlo sin depender de quienes lo escribieron.

Todo lo descripto surge del propio repositorio: el código de la app y del panel, las 83 migraciones SQL de la base de datos, los scripts de prueba, la documentación previa de la carpeta `docs/`, el informe de auditoría técnica del 13/09/2026 y el historial de commits.

Convenciones

- Los identificadores `RF-`, `RNF-`, `RN-`, `CU-` y `US-` son los mismos que usa la documentación de `docs/`, para poder trazar un requerimiento hasta su caso de uso, su historia y su prueba.
- Los nombres de tablas, funciones y archivos aparecen en `fuente monoespaciada`.
- El **Anexo A** reúne las sugerencias: lo que falta o conviene reforzar para cumplir la consigna por completo.

Versión	Fecha	Descripción
1.0	08/10/2026	Primera versión integral, elaborada a partir del código, las migraciones y la documentación del repositorio.
1.1	22/10/2026	Actualización con los últimos cambios (verificación de titularidad, re-revisión al editar, foto obligatoria, cierre de chats al eliminar una publicación y fotos de vehículos privadas; migraciones 0077–0082), datos del equipo y de la cursada, diagramas actualizados. Se quitó el manual de usuario integrado: se entrega como documento aparte.
1.2	23/10/2026	Capturas de la sección 8.1, resultados de las pruebas (scripts SQL y concurrencia 12/12), cuentas de demostración y migración 0083 (el mapa de Inicio muestra todos los estacionamientos publicados).

1. Presentación general del proyecto

1.1 Nombre del producto

GarageGo. Identificador de la aplicación: `com.garagego.app`; esquema de enlace profundo: `garagego://`. El nombre combina *garage* (cochera) y *go* (ir): encontrar un lugar y salir.

1.2 Integrantes y roles del equipo

Integrante	Rol principal	Responsabilidades
Bruno Bellot	Líder de proyecto / Product Owner	Prioriza el backlog, toma las decisiones de producto (por ejemplo, las cuestiones Q1–Q14 del flujo de pago) valida las entregas e integra el trabajo del equipo en el repositorio.
Ignacio Quintana	Desarrollo móvil (frontend)	Pantallas Expo/React Native, navegación, formularios y consumo de la API.
Nicolás Porcel	Backend y base de datos	Migraciones SQL, funciones PL/pgSQL, políticas RLS, tareas programadas.
Braian Carballo	Diseño UX/UI y panel de administración	Prototipo en Figma, sistema de diseño (tokens), accesibilidad y desarrollo del panel web de moderación, soporte y métricas.
Gianluca Polizzi	QA, seguridad y documentación	Casos de prueba, scripts de verificación, manuales y esta documentación.

1.3 Problema o necesidad identificada

En las zonas urbanas densas estacionar es **caro, lento e incierto**: el conductor no sabe si va a encontrar lugar hasta llegar, da vueltas a la manzana y termina pagando un estacionamiento comercial o dejando el auto lejos de su destino. Al mismo tiempo, **miles de cocheras particulares quedan vacías** gran parte del día (mientras el dueño trabaja, de vacaciones o porque no tiene auto) sin generar ningún ingreso.

No existía una forma simple y confiable de cruzar ambas puntas: el conductor necesita **asegurarse un lugar concreto para un horario concreto**, y el propietario necesita **cobrar sin gestionar nada a mano** y con reglas claras ante cancelaciones o demoras.

1.4 Objetivo general del sistema

Permitir que una persona **reserve y pague por adelantado** una cochera particular para un horario determinado, y que el dueño de esa cochera la **publique y la monetice** sin gestión manual, con un equipo de administración que modera las publicaciones y atiende el soporte.

El MVP se considera cumplido cuando un conductor completa el ciclo entero **buscar** → **reservar** → **pagar** → **estacionar** → **finalizar** → **calificar** y el propietario recibe el dinero correspondiente, sin intervención del equipo.

1.5 Público o tipo de usuario

Usuario	Perfil	Qué busca
Conductor	Persona que se mueve en auto por la ciudad (trabajo, trámites, salidas) y necesita estacionar por horas. Es el rol con el que nace toda cuenta.	Encontrar un lugar cerca de su destino, saber el precio antes de ir y tener el lugar asegurado.
Propietario	Particular con una **cochera de casa** o responsable de un **estacionamiento** (playa o predio con varios lugares). Usa la misma cuenta, activando el modo propietario.	Obtener ingresos de un espacio ocioso, decidir cuándo está disponible y cobrar automáticamente.
Administrador	Integrante del equipo de GarageGo. Opera desde el panel web, no desde la app.	Que solo se publiquen cocheras reales y completas, atender reclamos y seguir las métricas.

Contexto geográfico y cultural: **Argentina** (Ciudad de Buenos Aires como zona inicial). Por eso la interfaz está en español rioplatense, los importes en pesos argentinos, las patentes y teléfonos se validan con formato argentino y todos los horarios se interpretan en hora de Buenos Aires.

1.6 Alcance del producto

El producto entregado está compuesto por:

- **App GarageGo** (un solo código React Native + Expo) para **Android** (APK de distribución interna), **web** (publicada en Cloudflare) e **iOS** (compilable, no probada en dispositivo).
- **Panel de administración** web (React + Vite), solo para escritorio.
- **Backend en Supabase**: base PostgreSQL con toda la lógica de negocio, autenticación, almacenamiento de imágenes, tiempo real y tareas programadas.

El dinero se mueve dentro de una **billetera interna simulada**: el saldo se carga con un botón de prueba, sin cobro real. Todo el resto del ciclo (débito al reservar, reembolso al cancelar, cobro del propietario al finalizar, cargo por tiempo excedido) funciona sobre esa billetera con las mismas garantías que tendría con dinero real.

1.7 Funcionalidades incluidas y fuera del alcance

Incluidas

Área	Funcionalidades
Cuenta	Registro, verificación de correo, ingreso con correo o Google, recuperación de contraseña con código, perfil editable, una cuenta con dos roles (conductor / propietario).
Conductor	Mapa de Inicio con todos los estacionamientos publicados y pines que indican si la cochera está disponible ahora; búsqueda por ubicación o por dirección escrita, lista con filtros, ficha con galería, servicios y reseñas, ruta hasta la cochera, reserva con vehículo, pago con saldo, comprobante, inicio y fin del estacionamiento, cancelación, historial, calificación, hasta 3 vehículos (con fotos privadas).

Área	Funcionalidades
Propietario	Asistente de publicación en 5 pasos (datos del lugar, tarifa y servicios, disponibilidad, fotos y titularidad), dirección escrita que se ubica sola en el mapa, tipo de publicación (cochera de casa o estacionamiento), capacidad de 1 a 50 autos, servicios, horarios por día y días bloqueados, de 1 a 3 fotos distintas, verificación de titularidad con DNI y documentos, estado de revisión (una publicación aprobada que se edita vuelve a revisión), pausar / archivar / eliminar, reservas recibidas con detalle, dashboard y estadísticas.
Transversal	Chat en tiempo real por cochera, notificaciones por rol (con recordatorio 24 h antes y aviso 15 min antes del fin), centro de ayuda con tickets, cierre automático de reservas olvidadas, textos legales e información.
Administración	Moderación de publicaciones (aprobar, rechazar, pedir cambios indicando qué corregir) con revisión de los documentos de titularidad, que se borran al decidir; pausa y suspensión de cocheras, tickets de soporte, moderación de reseñas, dashboard de métricas, directorio y ficha de usuarios, listado de transacciones.

Fuera del alcance actual

Tema	Estado	Detalle
Pasarela de pago (Mercado Pago)	Diseñado, no implementado	Diseño cerrado el 17/09/2026: reservar primero en estado *pendiente* y confirmar por webhook, timeout de 10 minutos, reintentos y reembolso automático si el pago llega tarde. Requiere Edge Functions.
Carga de saldo con dinero real	No implementado	La recarga es una simulación marcada como `is_demo`, con topes (saldo máximo $200.000, recarga máxima $50.000) y desactivable por entorno.
Retiro de fondos del propietario	Descartado del MVP	El propietario acumula saldo pero no puede transferirlo a un banco.
Tarjetas guardadas	No implementado	La sección existe en la interfaz con la leyenda «Próximamente».
Notificaciones push remotas	Parcial	Hay avisos dentro de la app (en tiempo real) y notificaciones **locales** programadas en el celular; no hay push enviado desde el servidor.
Eliminación de cuenta	Parcial	El botón «Eliminar cuenta» existe en Ajustes pero todavía no tiene funcionalidad.
Modo oscuro	Parcial	La paleta oscura está definida pero no se activa.
Aprobación de la reserva por el propietario	Descartado	La reserva queda confirmada al pagarse; el propietario puede cancelarla.
Plazas numeradas, abonos mensuales, reservas recurrentes	Fuera del alcance	Cada reserva es un evento puntual sobre la cochera completa (con su capacidad).
Facturación fiscal, multi-idioma, multi-moneda, seguros	Fuera del alcance	El comprobante es solo en pantalla; solo español y pesos; los conflictos se canalizan por soporte.
Publicación en tiendas	Pendiente	La distribución actual es por APK y por la versión web.

2. Requerimientos

2.1 Requerimientos funcionales

Qué debe permitir hacer el sistema. Se agrupan por módulo.

Autenticación y cuenta

ID	Requerimiento
RF-A01	Registrarse con nombre, apellido, correo y contraseña.
RF-A02	Verificar la dirección de correo mediante un enlace.
RF-A03	Iniciar sesión con correo y contraseña, pudiendo ver u ocultar la contraseña.
RF-A04	Iniciar sesión con una cuenta de Google.
RF-A05	Recuperar la contraseña en tres pasos: correo, **código numérico** recibido por mail y nueva contraseña.
RF-A06	Cerrar sesión.
RF-A07	Mantener la sesión iniciada entre aperturas de la app.

Perfil, vehículos y billetera

ID	Requerimiento
RF-P01	Consultar y editar el perfil: nombre, apellido, teléfono y foto.
RF-P02	Alternar entre modo conductor y modo propietario sin cerrar sesión.
RF-P03	Registrar hasta **3 vehículos** (patente, marca y modelo obligatorios; color opcional) con fotos.
RF-P04	Consultar el saldo de la billetera.
RF-P05	Cargar saldo (simulación de pagos del MVP).
RF-P06	Consultar el historial de movimientos.

Conductor

ID	Requerimiento
RF-C01	Buscar cocheras cerca de la ubicación actual **o de una dirección escrita**, filtrando por distancia, precio máximo y calificación mínima.
RF-C02	Ver los resultados en un mapa, con un pin por cochera que distingue tipo (casa o estacionamiento) y **si está disponible ahora**.
RF-C03	Ver los resultados como lista.
RF-C04	Consultar la ficha de una cochera: galería de fotos, dirección, precio, horarios, capacidad, servicios, calificación y reseñas.
RF-C05	Ver la ruta desde la ubicación actual hasta la cochera.

ID	Requerimiento
RF-C06 / C07	Elegir fecha y horario; solo se ofrecen los horarios realmente libres.
RF-C08	Reservar indicando el vehículo; un vehículo no puede estar en dos reservas a la vez.
RF-C09 / C10	Pagar con el saldo y ver el comprobante en pantalla.
RF-C11	Consultar reservas activas y próximas, con cuenta regresiva y el vehículo de la reserva.
RF-C12 / C13	Iniciar el estacionamiento al llegar y finalizarlo al retirarse.
RF-C14	Cancelar una reserva dentro de las condiciones permitidas.
RF-C15 / C16	Consultar el historial y quitar reservas de él sin afectar a la otra parte.
RF-C17	Calificar la cochera (1 a 5 y comentario) si la reserva finalizó **con registro de entrada**.

Propietario

ID	Requerimiento
RF-O01 a O05	Publicar una cochera con nombre, descripción, dirección escrita que se ubica sola en el mapa, ubicación en el mapa, precio por hora, horarios por día y de 1 a 3 fotos distintas entre sí.
RF-O06	Enviar la publicación a revisión y conocer su estado (pendiente, aprobada, rechazada, cambios solicitados).
RF-O07	Editar y reenviar a revisión cuando se pidieron cambios; el sistema verifica que se hayan corregido los campos indicados.
RF-O08 / O09	Pausar, reactivar y eliminar una publicación (se archiva si tiene historial; al eliminarla se cierran sus chats y ya no admite reclamos).
RF-O10	Consultar las reservas recibidas con datos del conductor y su vehículo, y el **detalle** de cada una.
RF-O11 / O12	Finalizar una reserva en curso y cancelar una reserva recibida.
RF-O13	Cobrar automáticamente en la billetera al finalizar la reserva.
RF-O14 / O15	Consultar el dashboard de actividad e ingresos y las estadísticas por cochera.
RF-O16	Indicar la **capacidad** (cuántos autos entran a la vez, de 1 a 50).
RF-O17	Indicar los **servicios** (cámaras, vigilancia, iluminación, cargador eléctrico, sanitarios).
RF-O18	Indicar el **tipo de publicación**: cochera de casa o estacionamiento.
RF-O19	Bloquear días puntuales desde un calendario (excepciones de disponibilidad).
RF-O20	Acreditar la titularidad: indicar la relación con el lugar (titular, inquilino o administrador) y subir el DNI y los documentos que correspondan.
RF-O21	Al editar una publicación aprobada, volver a revisión si cambian los datos del lugar, la tarifa, los servicios o las fotos; cambiar la dirección exige acreditar la titularidad otra vez.

Transversales y administración

ID	Requerimiento
RF-T01	Chatear en tiempo real con la contraparte; una conversación por cochera.
RF-T02	Recibir notificaciones separadas por rol; la de mensaje indica quién escribió y de qué cochera.
RF-T03	Abrir tickets de soporte vinculados a una reserva, a una cochera o como **consulta general**, y conversar con el equipo.
RF-T04	Cerrar automáticamente las reservas que nadie finalizó.
RF-T05	Recordar la reserva 24 h antes y avisar 15 min antes del fin.
RF-AD01	Ingresar al panel solo con cuentas marcadas como administrador.
RF-AD02 / AD03	Aprobar, rechazar o pedir cambios (indicando los campos), revisar los documentos de titularidad antes de aprobar y pausar o suspender cocheras publicadas.
RF-AD04	Responder, resolver y reabrir tickets.
RF-AD05 a AD08	Consultar métricas, directorio y ficha de usuarios, ocultar reseñas y ver transacciones.
RF-AD09	Borrar los documentos de titularidad al aprobar o rechazar, dejando un sello de verificación.

2.2 Requerimientos no funcionales

ID	Atributo	Requerimiento	Cómo se cumple
RNF-01	Seguridad	Cada usuario solo lee y modifica sus datos.	Políticas RLS en las 18 tablas; nada depende de ocultar botones.
RNF-02	Seguridad	Ninguna credencial privilegiada viaja en el cliente.	Solo se distribuye la *anon key*; la `service_role` nunca se usa en la app ni en el panel.
RNF-03	Seguridad	Los permisos no pueden auto-asignarse.	Disparadores que congelan `is_admin` y el estado de moderación.
RNF-04	Integridad	Las operaciones de dinero y reserva son atómicas.	Funciones PL/pgSQL transaccionales; o se aplica todo o nada.
RNF-05	Concurrencia	Dos operaciones simultáneas no duplican cobros ni sobre-reservan.	`SELECT … FOR UPDATE`, *advisory locks* por cochera y por vehículo, índice único contable. Probado con 12 escenarios reales.
RNF-06	Compatibilidad	Android, iOS y navegador con un único código.	Expo + React Native Web; archivos `.web.tsx` solo para mapas, ubicación y geocodificación.
RNF-07	Usabilidad	Interfaz en español rioplatense, importes en pesos.	Textos propios; formato de moneda centralizado en `src/lib/money.ts`.
RNF-08	Accesibilidad	Contraste mínimo AA (4,5:1) en textos.	Tokens `onPrimary` y `accent` separados del lima de marca.

ID	Atributo	Requerimiento	Cómo se cumple
RNF-09	Rendimiento	Búsquedas y listados acotados.	Búsqueda server-side con PostGIS (radio máx. 1000 km desde la 0083 y hasta 200 resultados por pedido); listados con paginación por cursor; caché con TanStack Query.
RNF-10	Disponibilidad	Sin infraestructura propia que operar.	Supabase administrado; hosting web en Cloudflare.
RNF-11	Consistencia horaria	El resultado no depende de dónde esté el teléfono.	Todo se calcula en hora de Buenos Aires (`shared/time.ts` y funciones SQL); probado en 6 zonas horarias.
RNF-12	Mantenibilidad	Cambios de base versionados y re-ejecutables.	83 migraciones numeradas e idempotentes; tipos compartidos entre app y panel (`shared/domain.ts`).
RNF-13	Calidad	Verificación automática en cada cambio.	GitHub Actions: verificación de tipos, tests y tests de zona horaria en app y panel.
RNF-14	Trazabilidad	Todo movimiento de dinero queda registrado.	`wallet_transactions` con referencia a la reserva y marca `is_demo`.
RNF-15	Privacidad	Los datos personales sensibles se guardan solo mientras hacen falta y no son públicos.	Documentos de titularidad y fotos de vehículos en *buckets* privados con URLs firmadas que vencen; los documentos se borran al decidir la revisión.

2.3 Reglas de negocio principales

Todas las reglas se aplican **en el servidor**, dentro de funciones o políticas de PostgreSQL, por lo que se cumplen aunque alguien modifique la app o llame a la base directamente. El detalle completo (45 reglas, con la migración donde vive cada una) está en `docs/03-REGLAS-NEGOCIO.md`; estas son las centrales:

Regla	Descripción
Aprobación previa	Una cochera nueva queda *pendiente* y no aparece en las búsquedas hasta que el equipo la aprueba; para aprobarla debe tener al menos una foto y los documentos de titularidad completos. El rechazo es definitivo; los cambios solicitados se corrigen y se reenvían.
Titularidad	Quien publica indica su relación con el lugar y sube su DNI más: escritura o boleta del impuesto inmobiliario/ABL (titular); contrato de alquiler y autorización del dueño (inquilino); autorización del dueño (administrador). En estacionamientos puede sumar la habilitación municipal. Los documentos se borran al aprobar o rechazar y queda un sello de verificación.
Re-revisión al editar	Si una cochera aprobada cambia sus datos del lugar, tarifa, servicios, tipo o fotos, vuelve a *pendiente* y sale de la búsqueda hasta una nueva aprobación; la disponibilidad y la pausa no la revisan. Cambiar la dirección o el punto del mapa quita el sello de titularidad.
Visibilidad	Solo se ofrece lo aprobado, activo, no pausado ni suspendido ni archivado. Nadie ve ni reserva sus propias cocheras.

Regla	Descripción
Disponibilidad	El horario debe caber en una franja configurada (o en una excepción) y no superar la **capacidad** contando un **margen de 15 minutos** antes y después de cada reserva.
Duración	Mínimo 30 minutos, en tramos de 30. No se reserva el pasado (comparando en hora argentina).
Vehículo	Toda reserva declara un vehículo propio, que no puede estar en otra reserva superpuesta ni en otra sesión en curso.
Precio	`precio = precio por hora × horas`, redondeado a 2 decimales y **congelado** al reservar.
Pago	Se debita en la misma transacción que crea la reserva; con saldo insuficiente no se crea. La billetera nunca queda negativa.
Confirmación	La reserva queda confirmada al pagarse; el propietario no necesita aprobarla.
Inicio	El conductor puede iniciar desde 15 minutos antes del inicio y hasta el fin reservado.
Finalización y cobro	Pueden finalizar conductor o propietario. El propietario cobra el **precio pactado** al finalizar; el excedente lo retiene la plataforma.
Tiempo excedido	Tolerancia de 5 minutos; luego se cobra a **1,5 × el precio por hora**, con un tope de 2 horas. Si el saldo no alcanza, se cobra lo disponible.
Cierre automático	Cada minuto se cierran las reservas vencidas hace más de 15 minutos: sin entrada (no-show) el propietario cobra igual; con entrada y sin salida se cobran 15 minutos de excedente.
Cancelación	El conductor puede cancelar hasta 1 hora antes; el propietario en cualquier momento. Reembolso del 100 % y el horario se libera.
Calificación	Solo reservas propias, finalizadas y con registro de entrada; una por reserva, de 1 a 5. Ocultar una reseña no la saca del promedio.
Borrado de cochera	Con reservas vigentes no se puede; con historial se archiva (se borran sus chats y no se pueden abrir tickets sobre ella); sin reservas se borra.
Saldo de prueba	Recarga máxima de $50.000 por operación y saldo total máximo de $200.000.

2.4 Restricciones conocidas del proyecto

Tipo	Restricción
Presupuesto	Costo cero: el equipo decidió no contratar servicios pagos. Por eso la ruta usa OSRM (gratuito) y no Google Directions, y el pago es simulado.
Arquitectura	Sin servidor propio: toda la lógica vive en Supabase. Cualquier integración que necesite un secreto (como una pasarela de pago) exige sumar Edge Functions.
Operación	Las migraciones se ejecutan **a mano** en el SQL Editor de Supabase, en orden; no hay acceso directo a la base desde el entorno de desarrollo.
Pruebas en celular	Se necesita un *development build* de EAS: el ingreso con Google no funciona en Expo Go porque Supabase rechaza las URLs de retorno con IP numérica.
Servicios de terceros	OSRM es un servidor demo sin garantía de disponibilidad; las claves de Google Maps quedan visibles en el cliente y deben restringirse en Google Cloud.

Tipo	Restricción
Dominio	Una sola zona horaria (Buenos Aires), una sola moneda (pesos) y un solo idioma (español).
Distribución	APK de distribución interna y versión web; no hay publicación en tiendas.
Tiempo	El desarrollo se ajustó al calendario de la cursada (junio a octubre de 2026).

3. Usuarios, actores y casos de uso

3.1 Tipos de usuarios y actores

Actor	Tipo	Descripción
Visitante	Primario	Persona sin sesión: solo puede registrarse, iniciar sesión o recuperar la contraseña.
Conductor	Primario	Busca, reserva, paga, estaciona y califica. Rol por defecto de toda cuenta.
Propietario	Primario	Publica y administra cocheras, recibe reservas y cobra. Es la misma cuenta con otro rol activo.
Administrador	Primario	Modera publicaciones y reseñas, atiende soporte y consulta métricas desde el panel web.
Planificador (pg_cron)	Secundario (sistema)	Proceso programado en la base: cierra reservas vencidas, envía avisos y limpia excepciones pasadas.
Supabase Auth	Secundario (sistema)	Valida credenciales, envía correos y resuelve el ingreso con Google.
Google Maps / geocodificador	Secundario (sistema)	Dibuja el mapa y traduce direcciones a coordenadas.
OSRM	Secundario (sistema)	Calcula el trayecto hasta la cochera.
GPS del dispositivo	Secundario (sistema)	Provee la ubicación actual.

Solo el visitante, el conductor, el propietario y el administrador son tipos de usuario: personas que usan el sistema. Los actores secundarios no son usuarios sino procesos o sistemas externos que participan en los casos de uso sin iniciar sesión. El planificador (`pg_cron`) figura como actor porque inicia por sí mismo el cierre automático de las reservas vencidas y el envío de avisos, sin que ninguna persona lo pida.

3.2 Permisos y responsabilidades

Actor	Puede	No puede	Responsabilidad
Conductor	Ver cocheras aprobadas; reservar, pagar, iniciar, finalizar y cancelar sus reservas; chatear; calificar; abrir tickets; gestionar sus vehículos y su saldo.	Ver reservas, vehículos, billeteras o mensajes ajenos; reservar su propia cochera; cancelar dentro de la última hora; calificar sin haber usado el lugar.	Llegar en horario, finalizar al retirarse y respetar la tolerancia.

Actor	Puede	No puede	Responsabilidad
Propietario	Publicar y editar sus cocheras; definir disponibilidad, capacidad y precio; subir los documentos de titularidad; ver reservas recibidas con el vehículo del conductor; finalizar y cancelar; ver estadísticas.	Aprobarse a sí mismo; editar una publicación rechazada; borrar una cochera con reservas vigentes; ver vehículos que no estén en una reserva suya.	Mantener la información real y recibir al conductor; cancelar solo con motivo.
Administrador	Leer perfiles, cocheras y reseñas; moderar y revisar los documentos de titularidad, que se borran al decidir; responder tickets; consultar métricas, usuarios y transacciones mediante funciones específicas.	Escribir directamente sobre reservas o billeteras; asignarse permisos desde la app (la marca de administrador se otorga solo desde la consola de la base).	Revisar publicaciones con criterio, atender el soporte y cuidar los datos personales.
Planificador	Ejecutar la liquidación de reservas vencidas y generar avisos.	Actuar sobre reservas que no estén vencidas.	Garantizar que ninguna reserva quede abierta indefinidamente.

La autorización se aplica en la base: el rol activo de la app solo decide qué navegación se muestra. Ver la matriz completa de permisos en la sección 9.2.

3.3 Diagrama general de casos de uso

El diagrama de la página siguiente agrupa los casos de uso por actor. Los del grupo «Ambos roles» los ejecutan tanto el conductor como el propietario.

3.3 Diagrama general de casos de uso

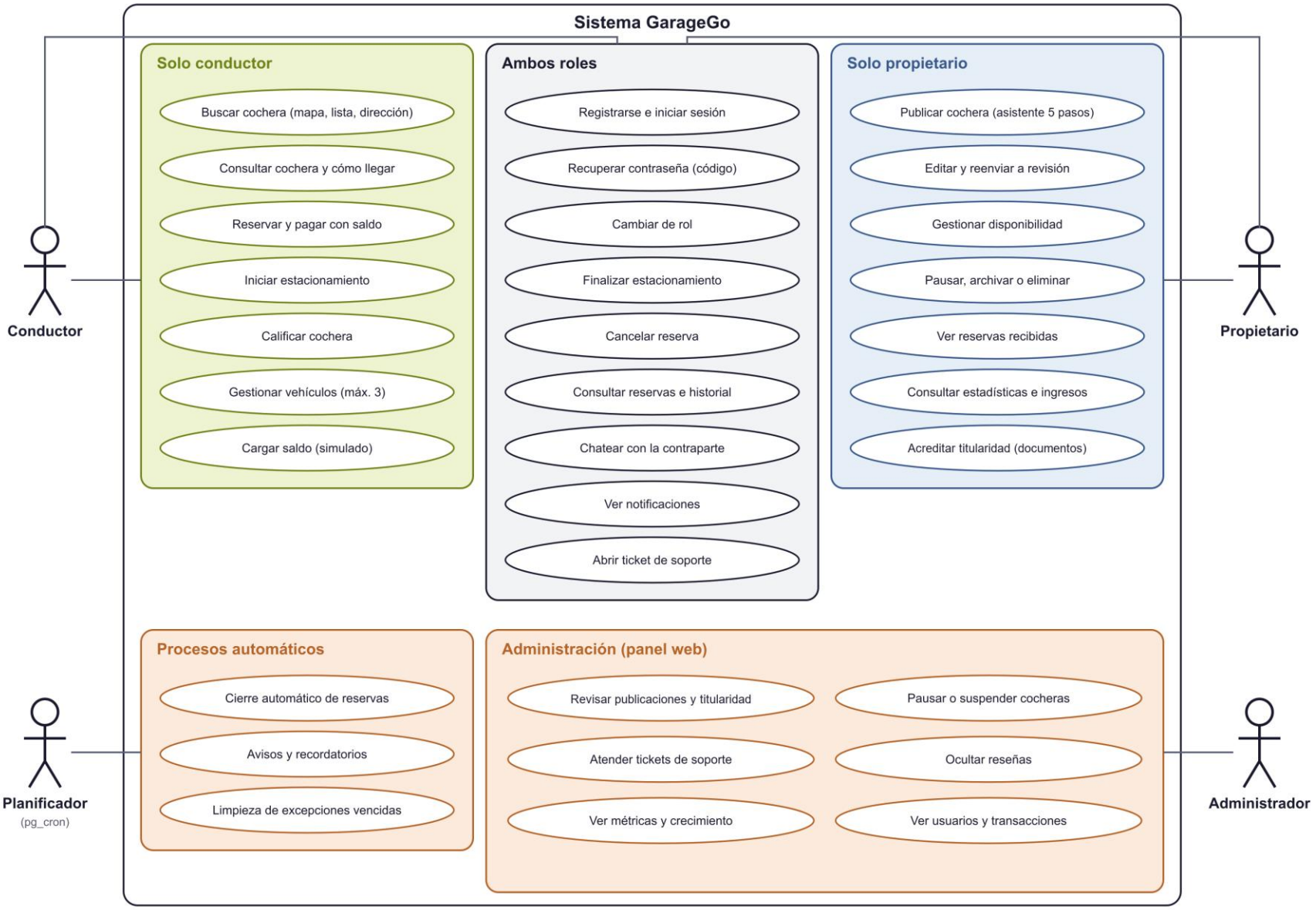


Figura 1. Diagrama de casos de uso de GarageGo, agrupados por actor.

3.4 Descripción de los casos de uso principales

Se detallan los casos de uso centrales del producto. Entre paréntesis se indica el código de error que devuelve el servidor en cada flujo alternativo; la app lo traduce a un mensaje comprensible.

CU-01 · Registrarse

Actor	Visitante (secundario: Supabase Auth)
Objetivo	Crear una cuenta para usar la aplicación.
Precondiciones	No tiene cuenta con ese correo.
Flujo principal	1. Abre «Registrarse» e ingresa nombre, apellido, correo, contraseña y su confirmación. 2. La app valida el formato (ver 8.4). 3. Supabase Auth crea la cuenta y envía el correo de verificación. 4. Un disparador crea el perfil (rol conductor) y la billetera en cero. 5. La app muestra la pantalla «Verificá tu correo».
Flujos alternativos	2a. Datos inválidos: se marca el campo y no se envía nada. 3a. Correo ya registrado: se informa y se ofrece iniciar sesión.
Resultado esperado	Cuenta creada con perfil y billetera; el usuario sabe que debe verificar el correo.

CU-02 · Iniciar sesión

Actor	Usuario registrado (secundario: Supabase Auth, Google)
Objetivo	Acceder a su cuenta en el rol con el que estaba operando.
Precondiciones	Tiene una cuenta.
Flujo principal	1. Ingresa correo y contraseña (puede mostrarla u ocultarla). 2. Supabase valida y devuelve la sesión. 3. La app asegura que exista el perfil (`ensure_profile`) y carga rol y saldo. 4. La guardia de navegación lo lleva al grupo de pantallas de su rol.
Flujos alternativos	1a. «Continuar con Google»: se abre el navegador del sistema (OAuth con PKCE) y vuelve a la app por `garagego://`. 2a. Credenciales incorrectas: mensaje genérico, sin revelar si el correo existe.
Resultado esperado	Sesión activa y persistente; el usuario ve la vista de su rol sin parpadeos.

CU-03 · Recuperar contraseña

Actor	Usuario registrado (secundario: Supabase Auth)
Objetivo	Definir una nueva contraseña sin perder la cuenta.
Precondiciones	Tiene una cuenta (también sirve para que una cuenta creada con Google defina una contraseña).

Flujo principal	1. Indica su correo. 2. Recibe un **código numérico** por mail. 3. Ingresa el código. 4. Define y confirma la nueva contraseña. 5. Queda con la sesión iniciada.
Flujos alternativos	1a. Correo inexistente: se muestra el mismo mensaje de confirmación (no se revela qué correos existen). 3a. Código incorrecto o vencido: se informa y puede pedir otro.
Resultado esperado	Contraseña reemplazada.

CU-04 · Buscar cochera

Actor	Conductor (secundarios: GPS, geocodificador, Google Maps)
Objetivo	Encontrar cocheras cerca de su ubicación o de su destino.
Precondiciones	Sesión iniciada como conductor.
Flujo principal	1. Abre «Inicio»; la app obtiene la ubicación del GPS y el mapa muestra todos los estacionamientos publicados. 2. Opcionalmente escribe una dirección de destino: el mapa encuadra las cocheras cercanas a ese lugar (o, si no hay, las 3 más cercanas) y muestra también el resto. 3. Desde «Buscar» puede limitar por distancia, precio máximo y calificación mínima. 4. El servidor (`search_parking_lots`, con PostGIS) devuelve las cocheras aprobadas y activas del área, sin las propias, ordenadas por cercanía. 5. El mapa muestra un pin por cochera (casa o estacionamiento; verde si está disponible ahora, gris si no). 6. Puede alternar a la vista de lista con los mismos filtros.
Flujos alternativos	1a. Permiso de ubicación denegado: se explica y se ofrece reintentar o buscar por dirección (en web se usa el Obelisco como punto por defecto). 4a. Sin resultados: se sugiere ampliar la distancia.
Resultado esperado	El conductor ve opciones reales y vigentes, con precio, para elegir una.

CU-06 · Reservar y pagar

Actor	Conductor
Objetivo	Asegurarse una cochera para una fecha y un horario.
Precondiciones	Tiene al menos un vehículo cargado y saldo suficiente.
Flujo principal	1. Desde la ficha elige fecha, hora de inicio y de fin; la app solo ofrece horarios libres. 2. Elige el vehículo. 3. Ve el total estimado y pasa a la pantalla de pago. 4. Confirma el pago con saldo. 5. `create_booking`, en una sola transacción: bloquea vehículo y cochera, valida disponibilidad, capacidad y margen, verifica el saldo, crea la reserva confirmada, debita y registra el movimiento, y notifica al propietario. 6. Se muestra el comprobante.

Flujos alternativos	2a. Sin vehículos: se lo deriva a cargar uno. 5a. Saldo insuficiente (`insufficient_balance`): no se crea nada; se ofrece cargar saldo. 5b. El horario se ocupó mientras decidía (`buffer_conflict`): se pide elegir otro. 5c. Horario fuera de la disponibilidad (`outside_availability`) o en el pasado (`booking_in_past`). 5d. El vehículo ya tiene otra reserva en ese horario (`vehicle_busy`). 5e. La cochera dejó de estar publicada (`lot_not_bookable`) o es propia.
Resultado esperado	Reserva confirmada, saldo debitado, propietario notificado y comprobante visible.

CU-07 · Iniciar estacionamiento

Actor	Conductor
Objetivo	Registrar la llegada real a la cochera.
Precondiciones	Reserva confirmada, sin iniciar, dentro de la ventana de inicio.
Flujo principal	1. Abre el detalle de la reserva y toca «Comenzar». 2. `start_booking` valida estado, ventana horaria y que el vehículo no esté en otra sesión. 3. Registra `actual_start_at` y notifica al propietario. 4. La pantalla muestra el anillo de progreso con el tiempo transcurrido y el restante.
Flujos alternativos	2a. Antes de 15 minutos del inicio (`start_too_early`). 2b. Pasado el fin reservado (`start_window_over`). 2c. Ya iniciada (`already_started`). 2d. El vehículo está en otra sesión en curso (`vehicle_in_use`).
Resultado esperado	Reserva en curso con la hora real de entrada.

CU-08 · Finalizar estacionamiento

Actor	Conductor, propietario o planificador
Objetivo	Cerrar la reserva, liquidar el excedente y pagar al propietario.
Precondiciones	Reserva confirmada **e iniciada** (el cierre manual exige registro de entrada).
Flujo principal	1. Se toca «Finalizar». 2. `complete_booking` bloquea la reserva y calcula el excedente sobre el fin pactado. 3. Si supera los 5 minutos de tolerancia, cobra el excedente (1,5 ×, tope 2 h) al conductor. 4. Acredita el precio pactado al propietario. 5. Marca la reserva como finalizada y notifica a ambos.
Flujos alternativos	1a. Nadie la finaliza: 15 minutos después del fin la cierra el planificador (no-show: sin excedente; olvidada: 15 minutos de excedente). 1b. Reserva no iniciada (`booking_not_started`). 3a. Saldo insuficiente para el excedente: se cobra lo disponible. 2a. Otra operación cambió la reserva al mismo tiempo (`booking_state_changed`): no se liquida dos veces.

Resultado esperado	Reserva finalizada, un único cobro al propietario y, si correspondía, un único cargo por excedente.

CU-10 · Cancelar reserva

Actor	Conductor o propietario
Objetivo	Anular una reserva y devolver el dinero.
Precondiciones	La reserva no está finalizada ni cancelada.
Flujo principal	1. Abre la reserva y elige «Cancelar»; un diálogo explica la consecuencia. 2. `cancel_booking` verifica que sea parte de la reserva y que esté permitido. 3. Marca la reserva como cancelada y reintegra el 100 % al conductor. 4. Notifica a la otra parte; el horario queda libre.
Flujos alternativos	2a. El conductor cancela dentro de la última hora (`cancel_too_late`): se rechaza con un aviso. 2b. Dos cancelaciones simultáneas: la segunda falla y hay un solo reembolso.
Resultado esperado	Reserva cancelada con reembolso único y horario liberado.

CU-14 · Publicar cochera

Actor	Propietario (secundario: administrador)
Objetivo	Ofrecer un espacio para que lo reserven.
Precondiciones	Modo propietario activo.
Flujo principal	1. **Datos del lugar**: nombre, descripción, dirección y punto exacto en el mapa. 2. **Tarifa y servicios**: tipo (casa o estacionamiento), precio por hora, capacidad y servicios. 3. **Disponibilidad**: franjas por día de la semana y días bloqueados en un calendario. 4. Fotos: de 1 a 3, distintas entre sí. 5. **Titularidad**: indica si es titular, inquilino o administrador y sube el DNI y los documentos que se le piden. 6. Se guarda en estado *pendiente* y queda en la cola de revisión del panel.
Flujos alternativos	2a. Precio no positivo o capacidad fuera de 1–50: error en el campo. 3a. Franja duplicada para el mismo día: se rechaza. 4a. Cuarta foto, archivo de más de 5 MB o formato no admitido: se rechaza. 4b. Sin fotos o con fotos repetidas: no puede avanzar (el servidor exige al menos una: `photo_required`). 5a. Faltan documentos de titularidad (`ownership_docs_required`): no se puede publicar hasta subirlos.
Resultado esperado	Publicación pendiente, no visible en búsquedas hasta su aprobación.

CU-15 · Corregir y reenviar una publicación

Actor	Propietario
Objetivo	Atender los cambios pedidos por el equipo.

Precondiciones	La publicación está en «cambios solicitados».
Flujo principal	1. Ve el motivo y los campos marcados por el equipo. 2. Edita la publicación en el asistente. 3. Al guardar, el servidor compara con la «foto» tomada al pedir los cambios. 4. Si se modificaron los campos pedidos, vuelve a *pendiente*.
Flujos alternativos	3a. Algún campo pedido sigue igual (`changes_not_made`): se informa cuál falta corregir. 1a. Publicación rechazada: no se puede editar, solo eliminar. 1b. Publicación ya aprobada: si cambian los datos del lugar, la tarifa, los servicios o las fotos, vuelve a revisión (la disponibilidad no); si cambia la dirección, debe acreditar la titularidad otra vez.
Resultado esperado	La publicación vuelve a la cola de revisión con las correcciones hechas.

CU-19 · Conversar por chat

Actor	Conductor y propietario
Objetivo	Coordinar la entrada, avisar demoras o resolver dudas.
Precondiciones	Existe al menos una reserva que vincula al conductor con la cochera.
Flujo principal	1. Abre «Mensajes» y elige la conversación de esa cochera (un hilo por cochera aunque haya varias reservas). 2. Escribe; el campo queda pegado al teclado. 3. El mensaje llega en tiempo real con fecha y hora, y la otra parte recibe una notificación con el nombre de quien escribió y la cochera.
Flujos alternativos	1a. Quien no es parte intenta leer el hilo: la base no devuelve nada.
Resultado esperado	Mensaje entregado y visible solo para las dos partes.

CU-20 · Abrir ticket de soporte

Actor	Conductor o propietario (secundario: administrador)
Objetivo	Pedir ayuda al equipo.
Precondiciones	Sesión iniciada.
Flujo principal	1. Entra a «Centro de ayuda» y crea un ticket. 2. Elige categoría y, opcionalmente, la reserva (conductor) o la cochera (propietario); si no elige ninguna, es una consulta general. 3. Describe el problema. 4. `create_support_ticket` crea el ticket (código `TCK-00042`) y su primer mensaje en una sola transacción. 5. Recibe una notificación cuando el equipo responde o resuelve.
Flujos alternativos	2a. Vínculo ajeno: se rechaza. 5a. Responde un ticket resuelto: se reabre.
Resultado esperado	Ticket visible para el equipo y para el usuario, en la pantalla del modo desde el que lo abrió.

CU-21 · Moderar publicación

Actor	Administrador
Objetivo	Evitar publicaciones falsas o incompletas.
Precondiciones	Hay publicaciones pendientes; sesión de administrador en el panel.
Flujo principal	1. Abre «Inventario → Aprobaciones». 2. Revisa datos, fotos, servicios, capacidad, disponibilidad y los documentos de titularidad, con una lista de control. 3. Aprueba, rechaza (con motivo) o pide cambios marcando los campos a corregir. 4. `admin_review_parking_lot` aplica el estado y notifica al propietario. 5. Al aprobar o rechazar, el panel borra los documentos y queda el sello de verificación; con «Pedir cambios» se conservan. 6. Si fue aprobada, aparece en las búsquedas.
Flujos alternativos	3a. Una cochera ya publicada con problemas: se pausa o suspende desde «Cocheras publicadas»; las reservas vigentes se mantienen. 3b. Faltan fotos o documentos (`photos_missing`, `ownership_docs_missing`): no se puede aprobar; se piden cambios.
Resultado esperado	Publicación con estado definido y propietario informado del motivo.

Otros casos de uso

Código	Caso de uso	Resumen
CU-05	Consultar cochera	Ficha con galería, servicios, capacidad, horarios, calificación, reseñas visibles y botón «Cómo llegar» (si OSRM falla, se ofrece abrir Google Maps).
CU-09	Consultar reservas	Activas y próximas con cuenta regresiva; historial separado, con opción de quitar una reserva solo para uno.
CU-11	Calificar cochera	Desde el historial; 1 a 5 y comentario; actualiza el promedio.
CU-12	Gestionar vehículos	Alta, edición, baja y fotos (privadas); máximo 3; patente argentina.
CU-13	Cargar saldo	Recarga simulada con topes; desactivable por entorno.
CU-16	Gestionar disponibilidad	Franjas por día (sin duplicados) y días bloqueados por calendario.
CU-17	Pausar, archivar o eliminar	Pausa/reactiva; borrar archiva si hay historial (y cierra sus chats) y se bloquea si hay reservas vigentes.
CU-18	Consultar reservas recibidas	Listado y detalle con conductor, vehículo y dirección; finalizar o cancelar.
CU-22	Atender ticket	Responder, resolver y reabrir desde el panel, con el contexto de la reserva o cochera.
CU-23	Cierre automático	El planificador cierra reservas vencidas y envía recordatorios y avisos.
CU-24	Acreditar titularidad	Paso 5 del asistente: relación con el lugar, DNI y documentos (imágenes); se vuelve a pedir si cambia la dirección.

4. Historias de usuario y planificación

4.1 Historias de usuario principales

Formato: *Como* [rol], *quiero* [acción], *para* [beneficio]. Prioridad según MoSCoW (M: imprescindible, S: importante, C: deseable). Las historias US-30 a US-39 se sumaron durante el desarrollo.

ID	Historia	Prior.	Estado
US-01	Como visitante quiero registrarme con mi correo para usar la app.	M	Hecha
US-02	Como usuario quiero iniciar sesión (correo o Google) para acceder a mi cuenta.	M	Hecha
US-03	Como usuario que olvidó su contraseña quiero restablecerla para no perder mi cuenta.	M	Hecha (falta SMTP propio)
US-04	Como usuario quiero editar mi perfil para mantener mis datos al día.	S	Hecha
US-05	Como usuario quiero pasar a modo propietario sin otra cuenta.	M	Hecha
US-06	Como conductor quiero buscar cocheras disponibles para encontrar dónde estacionar.	M	Hecha
US-07	Como conductor quiero ver las cocheras en un mapa para elegir por cercanía.	M	Hecha
US-08	Como conductor quiero ver la ficha de la cochera para decidir antes de reservar.	M	Hecha
US-09	Como conductor quiero ver la ruta hasta la cochera para llegar sin perderme.	S	Hecha
US-10	Como conductor quiero cargar mi vehículo para que el propietario sepa qué auto entra.	M	Hecha
US-11	Como conductor quiero reservar una cochera para asegurarme el lugar.	M	Hecha
US-12	Como conductor quiero pagar con mi saldo para dejar la reserva confirmada.	M	Hecha (saldo simulado)
US-13	Como conductor quiero indicar que llegué para registrar el inicio real.	M	Hecha
US-14	Como conductor quiero finalizar al retirarme para liberar la cochera.	M	Hecha
US-15	Como conductor quiero cancelar una reserva para recuperar mi dinero.	M	Hecha
US-16	Como conductor quiero ver mi historial para saber dónde estacioné y cuánto gasté.	S	Hecha
US-17	Como conductor quiero calificar la cochera para ayudar a otros.	S	Hecha
US-18	Como propietario quiero publicar mi cochera para ofrecerla.	M	Hecha
US-19	Como propietario quiero definir cuándo está disponible para que nadie reserve cuando la necesito.	M	Hecha
US-20	Como propietario quiero conocer el estado de mi publicación para entender por qué no recibo reservas.	M	Hecha

ID	Historia	Prior.	Estado
US-21	Como propietario quiero pausar, editar o eliminar mis cocheras.	S	Hecha
US-22	Como propietario quiero ver quién reservó y con qué vehículo.	M	Hecha
US-23	Como propietario quiero cobrar automáticamente cada reserva.	M	Hecha (sin retiro)
US-24	Como conductor o propietario quiero chatear con la otra parte para coordinar.	S	Hecha
US-25	Como usuario quiero recibir avisos de lo que pasa con mis reservas.	S	Hecha (sin push remoto)
US-26	Como usuario quiero pedir ayuda al equipo.	S	Hecha
US-27	Como administrador quiero revisar las publicaciones antes de que salgan.	M	Hecha
US-28	Como administrador quiero responder los tickets.	S	Hecha
US-29	Como administrador quiero ver métricas de la plataforma.	C	Hecha
US-30	Como propietario de un estacionamiento quiero indicar cuántos autos entran.	S	Hecha
US-31	Como propietario quiero mostrar los servicios y el tipo de mi cochera.	C	Hecha
US-32	Como conductor quiero un recordatorio el día anterior y un aviso antes del fin.	S	Hecha
US-33	Como conductor quiero saber en el mapa qué cocheras puedo usar ahora.	C	Hecha
US-34	Como usuario quiero hacer una consulta general sin elegir reserva ni cochera.	C	Hecha
US-35	Como administrador quiero indicar exactamente qué corregir al pedir cambios.	S	Hecha
US-36	Como usuario quiero eliminar mi cuenta.	S	Pendiente (botón sin función)
US-37	Como propietario quiero acreditar que el lugar es mío o que tengo permiso para ofrecerlo.	M	Hecha
US-38	Como administrador quiero revisar los documentos de titularidad antes de aprobar una publicación.	M	Hecha
US-39	Como conductor quiero que una cochera publicada siga siendo lo que el equipo aprobó, aunque el propietario la edite.	S	Hecha

4.2 Criterios de aceptación

Criterios de las historias centrales del ciclo de reserva. Cada criterio es verificable y tiene su prueba asociada en la sección 10.

US-01 · Crear una cuenta

- Pide nombre, apellido, correo, contraseña y confirmación.

- Rechaza una contraseña de menos de 8 caracteres o sin mayúscula, minúscula y número.
- Rechaza un correo ya registrado con un mensaje claro.
- Envía un correo de verificación y crea perfil y billetera en cero.

US-06 · Buscar cocheras

- Parte de la ubicación del GPS o de una dirección escrita; si el permiso fue denegado, lo informa y permite reintentar.
- Filtra por distancia, precio máximo y calificación mínima, en mapa o lista.
- No muestra cocheras propias, pausadas, suspendidas, archivadas ni pendientes.

US-11 / US-12 · Reservar y pagar

- Solo ofrece horarios libres dentro de la disponibilidad, respetando capacidad y margen de 15 minutos.
- Exige un vehículo propio que no esté en otra reserva superpuesta.
- No permite reservar el pasado; la duración mínima es 30 minutos.
- Muestra el total antes de confirmar; lo debita en el mismo acto; si no alcanza el saldo, no crea la reserva.
- La reserva queda confirmada sin aprobación y el propietario recibe una notificación.

US-13 · Iniciar el estacionamiento

- Habilitado desde 15 minutos antes del inicio y hasta el fin reservado.
- No puede iniciarse dos veces ni con un vehículo que esté en otra sesión.
- Muestra tiempo transcurrido y restante; notifica al propietario.

US-14 · Finalizar

- Pueden finalizar conductor y propietario, solo si la reserva fue iniciada.
- Cobra el excedente pasada la tolerancia de 5 minutos, a 1,5 × con tope de 2 horas.
- Acredita al propietario el precio pactado una sola vez, aunque dos personas finalicen a la vez.
- Si nadie finaliza, el sistema la cierra 15 minutos después del fin.

US-15 · Cancelar

- El conductor puede cancelar con más de 1 hora de anticipación; el propietario siempre.
- Devuelve el 100 % una sola vez, notifica a la otra parte y libera el horario.

US-18 / US-20 · Publicar y conocer el estado

- El asistente tiene 5 pasos y valida cada uno antes de avanzar.
- Entre 1 y 3 fotos distintas, de hasta 5 MB (jpeg, png o webp).
- La publicación nueva queda pendiente y no aparece en las búsquedas.
- Cada cambio de estado de la revisión genera una notificación con el motivo.

- Pide el DNI y los documentos que corresponden a la relación con el lugar; sin ellos no se puede publicar ni aprobar.
- Editar una publicación aprobada (datos, tarifa, servicios o fotos) la devuelve a revisión.

US-27 · Revisar publicaciones

- Lista las pendientes con todos sus datos y fotos.
- Permite aprobar, rechazar o pedir cambios indicando campos y motivo.
- Muestra los documentos de titularidad con una lista de control y los borra al aprobar o rechazar.
- Solo una cuenta con `is_admin` puede operar; la base lo vuelve a verificar en cada función.

Los criterios del resto de las historias están en `docs/02-HISTORIAS-USUARIO.md`.

4.3 Backlog del producto

Épica	Ítem	Prior.	Estado
Cuenta	Registro, login, Google, recuperación, perfil, roles	M	Hecho
Cuenta	SMTP propio para la plantilla del código de recuperación	S	Pendiente
Cuenta	Eliminación de cuenta (requisito legal y de tiendas)	S	Pendiente
Búsqueda	Mapa, lista, filtros, dirección, disponible ahora	M	Hecho
Reservas	Disponibilidad, capacidad, margen, vehículo, pago, comprobante	M	Hecho
Reservas	Sesión activa, finalizar, excedente, cancelación, historial	M	Hecho
Reservas	Cierre automático, recordatorios, notificaciones locales	S	Hecho
Pagos	Billetera interna simulada con topes y marca demo	M	Hecho
Pagos	Integración con Mercado Pago (Edge Functions + webhook)	S	Diseñado, sin implementar
Pagos	Retiro de fondos del propietario	C	Descartado para el MVP
Publicaciones	Asistente de 5 pasos, servicios, tipo, fotos, excepciones	M	Hecho
Publicaciones	Moderación con cambios solicitados validados	M	Hecho
Publicaciones	Verificación de titularidad, re-revisión al editar, al menos una foto	M	Hecho
Privacidad	Fotos de vehículos privadas y borrado de documentos al decidir	S	Hecho
Comunicación	Chat por cochera, notificaciones por rol, soporte por tickets	S	Hecho
Comunicación	Notificaciones push remotas	C	Pendiente
Administración	Panel: inventario, soporte, dashboard, usuarios, transacciones	M	Hecho
Calidad	Correcciones de la auditoría técnica (críticos, altos y backlog)	M	Hecho

Épica	Ítem	Prior.	Estado
Calidad	CI, tests unitarios, tests SQL, tests de concurrencia	S	Hecho
Calidad	Tests de interfaz / end-to-end	C	Pendiente
Calidad	Observabilidad (logs y métricas de errores)	C	Pendiente
Diseño	Rediseño según Figma, tokens, contraste AA	S	Hecho
Diseño	Modo oscuro y auditoría con lector de pantalla	C	Pendiente
Distribución	APK interno y versión web en Cloudflare	S	Hecho
Distribución	Publicación en Play Store / App Store	C	Pendiente

4.4 Priorización de funcionalidades

Se usó el método **MoSCoW** con tres criterios, en este orden:

1. **Cerrar el ciclo completo antes de agregar variantes.** Primero buscar → reservar → pagar → estacionar → finalizar → calificar, con el cobro del propietario; recién después capacidad, servicios o recordatorios.
2. **La integridad del dinero y de las reservas antes que las funcionalidades nuevas.** Cuando la auditoría técnica detectó que se podían alterar reservas y duplicar reembolsos, se frenó el desarrollo de funcionalidades hasta corregir los hallazgos críticos y altos.
3. **Costo cero y bajo riesgo.** Se prefirieron soluciones sin costo (OSRM, billetera simulada) y se postergaron las que requieren infraestructura nueva (Edge Functions para Mercado Pago, push remoto).

4.5 Registro de tareas realizadas, pendientes y descartadas

Realizadas (síntesis)

- Fases originales 1–8: configuración, autenticación, perfil y billetera, módulo propietario, búsqueda, reservas, calificaciones, estadísticas y chat.
- Fases A–E: vehículos, selección de vehículo e inicio de reserva, notificaciones de mensajes, ajustes y ruta dibujada con OSRM.
- Rediseño completo según Figma (tokens, componentes, pantallas de los tres grupos).
- Flujo de pago y sesión activa (14 decisiones Q1–Q14), historial separado, notificaciones por rol.
- Panel de administración completo, primero con datos simulados y luego conectado a la base.
- Corrección de los hallazgos de la auditoría técnica (migraciones 0046–0061) y backlog técnico: React Query, paginación por cursor, CI, tests, zona horaria única.
- Funcionalidades finales: capacidad, servicios, tipo de publicación, recordatorios, cierre automático a los 15 minutos, un vehículo por reserva, galería, textos legales, asistente de publicación (primero en 4 pasos y luego en 5, con titularidad), disponible ahora, pedir cambios validados, tickets generales.

- Despliegue de la versión web y del panel en Cloudflare Workers.
- Restricción de la clave de Google Maps de Android por nombre de paquete y huella SHA-1.
- Verificación de titularidad y cierre del ciclo de la publicación (migraciones 0077–0082): documentos privados que se borran al decidir, re-revisión al editar una publicación aprobada, al menos una foto, cierre de chats al eliminar una publicación y fotos de vehículos privadas.
- Mapa de Inicio con todos los estacionamientos publicados; la distancia solo se limita desde «Buscar» (migración 0083).
- Todo el código commiteado y publicado en la rama `feature/ultimas-funcionalidades` (16/10/2026).

Pendientes

- Integración con Mercado Pago (diseño cerrado).
- Eliminación de cuenta, SMTP propio, notificaciones push remotas.
- Cloudflare Access para el panel, sesión en almacenamiento cifrado.
- Tests de interfaz y end-to-end, observabilidad, modo oscuro, auditoría de accesibilidad.

Descartadas

Tarea	Motivo
Aprobación de cada reserva por el propietario	Agregaba espera e incertidumbre al conductor; se reemplazó por confirmación inmediata con derecho a cancelar.
Google Directions para la ruta	Es pago y requiere facturación; se eligió OSRM por costo cero.
Comprobante en PDF y factura por correo	Fuera del alcance del MVP; el comprobante es en pantalla.
Plazas o zonas numeradas	Era contenido del mockup; la cochera se reserva como unidad.
Retiro de fondos	Implica transferir dinero real a terceros; queda fuera del MVP.
«Horario especial» en las excepciones	Se simplificó a días bloqueados elegidos en un calendario.
Usuarios en línea en el panel	El equipo decidió no registrar presencia de los usuarios.

4.6 Evolución del proyecto durante la cursada

Fecha	Hito	Migraciones
Hasta el 22/06	**MVP inicial.** Autenticación, perfil y billetera, publicaciones, búsqueda, reservas, calificaciones, chat, vehículos, inicio de reserva, notificaciones de mensajes y ajustes.	0001–0023
15/07	**Ruta y build propio.** Repositorio actual alineado con Expo SDK 57, mapa y chat en tiempo real, ruta dibujada con OSRM (costo cero) y *development build* de EAS en lugar de Expo Go (requisito del ingreso con Google).	—

Fecha	Hito	Migraciones
05/08	**Rediseño.** Pantallas, componentes y tokens según Figma; correcciones de un documento de QA del equipo.	—
07/09	**Flujo de pago.** Decisiones Q1–Q14, pago con saldo, sesión activa, cierre automático, historial y notificaciones por rol. Se corrige el error por el que el propietario no cobraba.	0024–0032
13/09	**Panel de administración.** Primero con datos simulados y luego conectado a la base: moderación, soporte, dashboard, usuarios, reseñas y transacciones.	0033–0045
24/09	**Auditoría técnica.** Revisión integral con Codex y corrección de todos los hallazgos (seguridad, concurrencia, permisos), más el backlog: React Query, paginación, CI, tests, zona horaria y Realtime en el panel. Diseño de la integración con Mercado Pago.	0046–0061
15/10	**Versión final.** Capacidad, servicios, recordatorios, cierre a los 15 minutos, un vehículo por reserva, galería, textos legales, disponible ahora, consultas generales, verificación de titularidad, re-revisión al editar y foto obligatoria. Despliegue de la web y del panel en Cloudflare.	0062–0082
23/10	**Cierre.** El mapa de Inicio muestra todos los estacionamientos; pruebas finales (scripts SQL y concurrencia 12/12), capturas y documentación.	0083

5. Diseño y arquitectura de la solución

5.1 Diagrama general de arquitectura

GarageGo tiene **dos clientes** (la app y el panel) y **un único backend administrado** (Supabase). La característica que define la arquitectura es que **no existe un servidor propio**: los clientes hablan directamente con Supabase, y toda la lógica de negocio vive dentro de la base de datos, escrita en PL/pgSQL.

5.1 Diagrama general de arquitectura

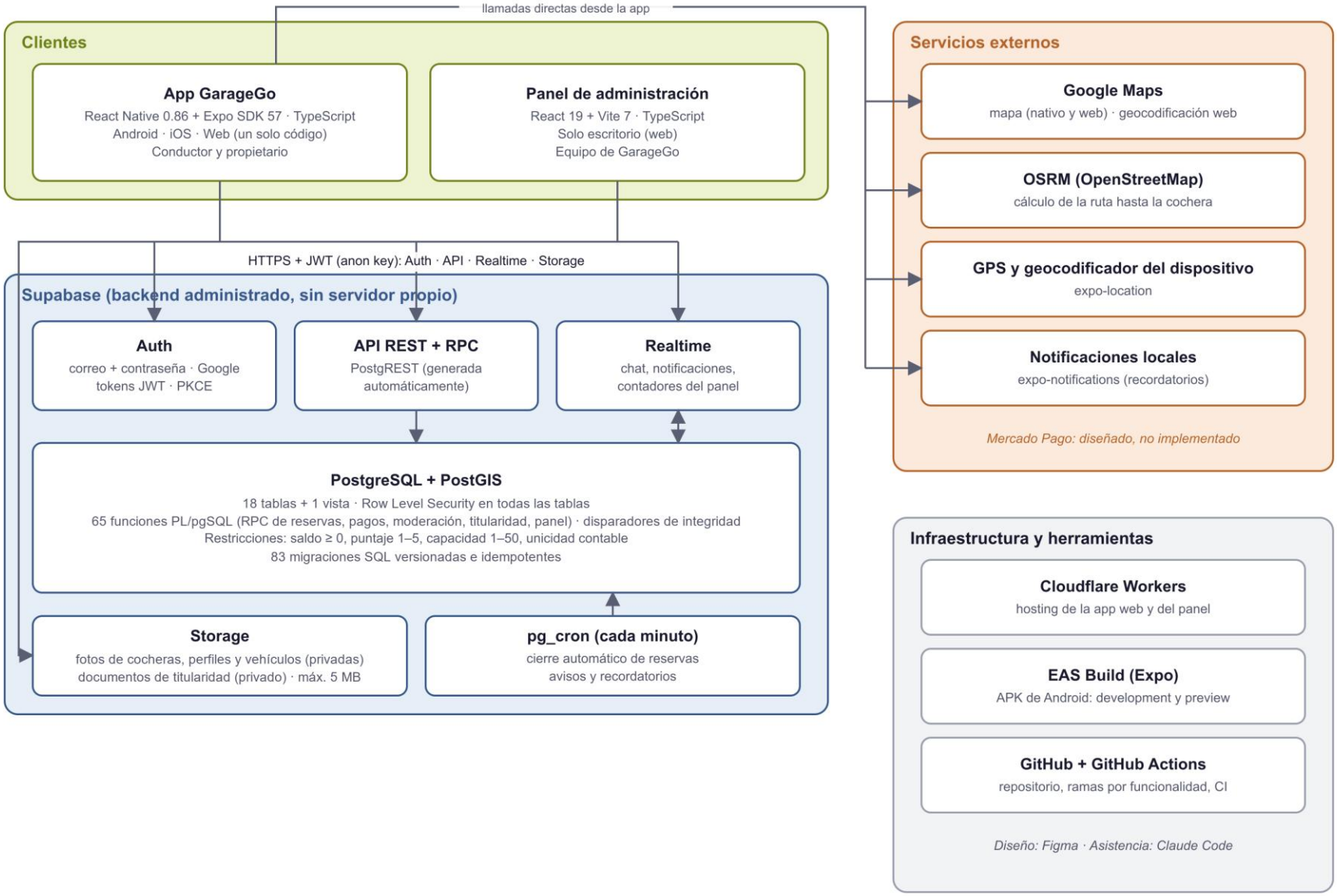


Figura 2. Arquitectura de GarageGo: clientes, backend Supabase, servicios externos e infraestructura.

5.2 Frontend, backend, base de datos, APIs y servicios externos

Capa	Componente	Tecnología	Responsabilidad
Frontend	App GarageGo	React Native + Expo (Android, iOS, web)	Experiencia del conductor y del propietario; navegación por rol; formularios; mapas; caché de datos.
Frontend	Panel de administración	React + Vite (web de escritorio)	Moderación, soporte, métricas, usuarios y transacciones.
Backend	Supabase Auth	GoTrue (servicio de Supabase)	Registro, login, Google OAuth con PKCE, verificación y recuperación por correo, emisión de tokens JWT.
Backend / API	API REST y RPC	PostgREST (generada desde el esquema)	Lecturas de tablas protegidas por RLS y llamadas a funciones (`create_booking`, `cancel_booking`, `admin_*`, …).
Backend	Realtime	Supabase Realtime (WebSocket)	Empuja mensajes de chat, notificaciones y contadores del panel sin consultar en bucle.
Backend	Storage	Supabase Storage	Fotos de cocheras y perfiles (públicas); fotos de vehículos y documentos de titularidad (privados, con URLs firmadas). 5 MB máx., jpeg/png/webp.
Backend	Tareas programadas	pg_cron	Cierre automático, avisos y recordatorios (cada minuto); limpieza diaria de excepciones vencidas.
Base de datos	PostgreSQL + PostGIS	18 tablas, 1 vista, 65 funciones	Datos, reglas de negocio, integridad y control de acceso.
Servicio externo	Google Maps Platform	Maps SDK (Android) / Maps JavaScript API (web)	Mapa, marcadores y geocodificación en la web.
Servicio externo	OSRM	API HTTP pública (OpenStreetMap)	Trazado de la ruta hasta la cochera.
Servicio del dispositivo	Ubicación y notificaciones	expo-location, expo-notifications	GPS, geocodificación nativa y recordatorios locales.
Infraestructura	Hosting y distribución	Cloudflare Workers, EAS Build, GitHub	Publicación web, generación de APK, repositorio y CI.

5.3 Comunicación entre los componentes

- **App y panel → Supabase:** mediante la biblioteca `@supabase/supabase-js` sobre HTTPS. Cada pedido lleva la *anon key* (que identifica al proyecto) y el **token JWT** del usuario, con el que la base aplica las políticas de acceso.
- **Dos formas de hablar con la base:** consultas directas a tablas para lecturas simples (listar mis vehículos, leer mi perfil) y **llamadas a funciones (RPC)** para toda operación que cambie algo importante o combine varias tablas (reservar, cancelar, finalizar, moderar).

- **Tiempo real:** el cliente se suscribe por WebSocket a los cambios de `messages`, `notifications`, `parking_lots` y `support_tickets`; Realtime aplica las mismas políticas de acceso que una consulta normal.
- **Archivos: las fotos y documentos se suben a Storage y en la tabla se guarda solo la ruta; los privados (fotos de vehículos y documentos de titularidad) se leen con URLs firmadas que vencen.**
- **Servicios externos:** la app consulta OSRM y Google Maps directamente (no hay secretos involucrados).
- **Retorno desde el navegador:** el ingreso con Google vuelve a la app por el enlace profundo `garagego://`, resuelto por expo-router.
- **Procesos sin usuario:** pg_cron ejecuta funciones dentro de la base; los resultados (por ejemplo, una notificación) llegan a la app por Realtime.

Recorrido de una reserva por todas las capas

1. El conductor elige cochera, fecha, horario y vehículo; la app calcula los horarios libres y el total estimado (`availability.ts`, `pricing.ts`).
2. Al confirmar el pago, la app invoca la RPC `create_booking`.
3. La función toma un *lock* por vehículo y otro por cochera, valida disponibilidad, capacidad, margen y saldo, crea la reserva confirmada, debita la billetera, registra el movimiento y crea la notificación del propietario. Todo en **una sola transacción**.
4. La app recibe el identificador y muestra el comprobante; el propietario recibe la notificación por Realtime.
5. Si cualquier paso falla, la transacción entera se deshace: no queda reserva, ni débito, ni notificación.

5.4 Decisiones de diseño relevantes

Decisión	Alternativa descartada	Justificación
React Native + Expo	Desarrollo nativo separado (Kotlin / Swift) o Flutter	Un solo código para Android, iOS y web con un equipo chico; EAS compila en la nube (APK sin depender de cada máquina, iOS sin Mac); módulos `expo-*` para ubicación, cámara, notificaciones y navegador.
expo-router (ruteo por archivos)	React Navigation configurado a mano	La estructura de carpetas es la navegación; resuelve enlaces profundos (retorno de Google, abrir la pantalla de una notificación); los grupos `(auth)`, `(client)` y `(owner)` cambian toda la navegación según el rol.
Supabase (PostgreSQL)	Firebase (Firestore)	El problema central —que dos personas no reserven el mismo lugar— es relacional y transaccional: rangos de tiempo, transacciones que reservan y debitan a la vez, restricciones como «saldo ≥ 0», consultas espaciales. Además aporta auth, storage, tiempo real y cron sin servidores propios.
Lógica de negocio en la base	Lógica en la app o en un servidor Node	Las operaciones críticas tocan varias tablas y deben ser indivisibles; en la app, un corte de red dejaría estados imposibles. Las reglas no se pueden esquivar modificando el cliente.

Decisión	Alternativa descartada	Justificación
Panel separado (Vite)	Pantallas de administración dentro de la app	Es otro producto (escritorio, tablas densas); evita cargar código inútil en el bundle móvil. Su autoridad viene de `is_admin`, no del código del panel.
OSRM para rutas	Google Directions API	Gratuito y sin clave. Se asume que es un servidor demo sin garantías y se mitiga con «Abrir en Google Maps».
Billetera interna simulada	Pasarela de pago desde el inicio	Permite completar y probar todo el ciclo del dinero sin costos ni secretos; el diseño de Mercado Pago quedó cerrado para la etapa siguiente.
Reservar y pagar en una sola operación	Crear la reserva y cobrar después	Evita reservas impagas bloqueando horarios.
Capacidad con locks, no con restricción de exclusión	Mantener `bookings_no_overlap`	Una restricción de exclusión solo expresa «ninguna superposición», no «menos de N»; el *advisory lock* por cochera serializa las reservas y la ocupación se mide como máximo simultáneo.
Zustand + TanStack Query	Un único estado global	La sesión y el rol son estado de la app; los datos del servidor son una copia con caché, revalidación y paginación.
Horarios sin zona + hora argentina	Guardar todo en UTC	«El martes de 9 a 12» significa eso en Buenos Aires; los instantes reales (`actual_start_at`) sí llevan zona. Todas las comparaciones se hacen en hora argentina.

5.5 Patrones y criterios de arquitectura

Patrón / criterio	Dónde se aplica
Cliente + *Backend as a Service*	Toda la solución: no hay capa intermedia propia.
Organización por dominio (*feature folders*)	`src/features/<dominio>/` con `api.ts`, `types.ts`, `schemas.ts` y pantallas propias.
Capa de acceso a datos por dominio (tipo *repository*)	Las pantallas no consultan la base directamente: siempre pasan por el `api.ts` de su dominio.
Máquina de estados	Reservas (`confirmed → en curso → completed / cancelled`) y publicaciones (`pending → approved / rejected / changes_requested; una aprobada que se edita vuelve a pending`).
Transacciones atómicas y bloqueo pesimista	`SELECT … FOR UPDATE` sobre la reserva y *advisory locks* por cochera y por vehículo.
Idempotencia	Actualización condicionada al estado esperado + índice único de movimientos por reserva y tipo.
Defensa en profundidad	RLS + funciones `security definer` acotadas + disparadores que protegen columnas sensibles.
Resolución por plataforma	Archivos `.web.tsx` para mapas, ubicación y geocodificación.
Sistema de diseño con tokens	`src/theme/` (colores, tipografía, espaciado, radios, sombras).

Patrón / criterio	Dónde se aplica
Paginación por cursor (*keyset*)	Listados de reservas, notificaciones, conversaciones, tickets y reseñas (migración 0059).
Migraciones idempotentes	Cada archivo puede re-ejecutarse sin romper nada (`IF NOT EXISTS`, `CREATE OR REPLACE`).
Tipos compartidos	`shared/domain.ts` y `shared/time.ts` usados por la app y por el panel.

5.6 Organización del código del cliente

Carpeta	Contenido
`app/`	Rutas y pantallas (expo-router): `(auth)`, `(client)` y `(owner)`. La estructura de carpetas es la navegación.
`src/features/`	13 dominios: auth, booking, directions, info, messaging, notifications, parking, profile, rating, search, stats, support, vehicle.
`src/components/`	27 componentes de interfaz genéricos (`ui/`) y componentes de dominio (`booking/`, `parking/`).
`src/lib/`	Cliente de Supabase, tipos generados de la base, dinero, paginación, claves de caché, mensajes de error.
`src/store/`	Estado global de sesión (Zustand).
`src/theme/`	Tokens de diseño.
`shared/`	Dominio y manejo de horas compartidos con el panel.

6. Tecnologías utilizadas

Para cada tecnología se indica versión (cuando es relevante), finalidad, motivo de elección, ventajas encontradas y dificultades o limitaciones detectadas durante el desarrollo.

6.1 Lenguajes

Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
TypeScript ~6.0 (app) / ~5.9 (panel)	Lenguaje de la app, el panel y los tipos compartidos.	Tipado estático sobre JavaScript; es el estándar de Expo y React.	Detecta errores antes de ejecutar; tipos de la base generados; contratos compartidos app–panel.	Los tipos de la base hay que regenerarlos o actualizarlos a mano al cambiar el esquema.
SQL / PL/pgSQL	Esquema, funciones de negocio, disparadores y políticas.	La lógica crítica debía ser transaccional y vivir en la base.	Atomicidad, restricciones verificadas por el motor, reglas imposibles de esquivar.	Menos familiar que TypeScript; más difícil de probar y depurar; los parámetros del negocio requieren una migración para cambiar.
JavaScript (Node.js)	Scripts de pruebas de concurrencia y de zona horaria.	Ya disponible en el entorno; acceso directo a Postgres con `pg`.	Permite abrir conexiones simultáneas reales.	Requiere la cadena de conexión de la base (dato sensible).
CSS	Estilos del panel.	Sin biblioteca de UI para mantener el panel liviano.	Control total; variables CSS como tokens.	Más trabajo manual en componentes complejos.

6.2 Frameworks y bibliotecas de la app

Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
React Native 0.86 + **React** 19.2	Base de la interfaz nativa.	Un código para varias plataformas con componentes nativos.	Ecosistema amplio; nueva arquitectura habilitada.	Actualizaciones de versión delicadas (reanimated y worklets tuvieron que fijarse a versiones exactas).
Expo SDK 57	Herramientas, módulos nativos y compilación.	Evita escribir código nativo; EAS Build.	Ubicación, cámara, notificaciones y fuentes listos para usar.	El login con Google no funciona en Expo Go: obliga a un *development build*.
expo-router 57	Navegación por archivos.	Enlaces profundos automáticos y grupos por rol.	La estructura de carpetas documenta la navegación.	Reemplazar un archivo por una carpeta del mismo nombre dejó pantallas duplicadas en una etapa.

Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
React Native Web 0.21	Versión web de la app.	Reutilizar el mismo código en el navegador.	Demo pública sin instalar nada.	Diferencias puntuales (mapas, ubicación, botones anidados) obligaron a archivos `.web.tsx`.
TanStack Query 5	Datos del servidor: caché, revalidación, paginación.	Evitar reescribir estados de carga y reintentos.	Pantallas más simples; paginación infinita.	Migración progresiva pantalla por pantalla.
Zustand 5	Estado global de sesión, perfil, rol y saldo.	Mínimo y sin *boilerplate*.	Lectura simple desde cualquier pantalla.	Hubo que cuidar respuestas viejas al cambiar de usuario (ciclo de vida de la sesión).
react-hook-form 7 + **zod** 4	Formularios y validación.	Validación tipada y reutilizable.	Esquemas probados con tests unitarios.	Las reglas deben coincidir con las del servidor (se duplican a propósito).
react-native-maps 1.27 / **@react-google-maps/api** 2.20	Mapa en nativo y en web.	Calidad de Google Maps en Argentina.	Marcadores personalizados por tipo y disponibilidad.	Dos implementaciones distintas; claves visibles en el cliente.
expo-location, **expo-image-picker**, **expo-notifications**, **expo-web-browser**	GPS y geocodificación, fotos, recordatorios locales, OAuth.	Módulos oficiales de Expo.	Integración directa.	Las notificaciones locales requieren un build nuevo para probarse.
react-native-keyboard-controller 1.21	Mantener el campo del chat pegado al teclado.	Comportamiento consistente en Android e iOS.	Mejor experiencia en el chat.	Requiere recompilar el build nativo.
date-fns 4	Fechas y cuentas regresivas.	Liviana y modular.	Formatos en español.	Toda la lógica horaria se centralizó luego en `shared/time.ts` para no depender del huso del teléfono.

6.3 Panel de administración

Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
React 19 + **Vite** 7 + **react-router-dom** 7	Aplicación web del backoffice.	Vite es mucho más liviano que Metro para un proyecto solo web.	Arranque y recarga instantáneos; build estático.	Metro detectaba las dependencias del panel y duplicaba React: se resolvió con `metro.config.js`.

6.4 Backend y base de datos

Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
Supabase (`supabase-js` 2.112)	Backend completo: auth, API, realtime, storage.	Todo lo necesario sobre PostgreSQL, sin operar servidores.	Velocidad de desarrollo; RLS nativo.	Dependencia del proveedor; las migraciones se corrieron a mano.
PostgreSQL (administrado por Supabase)	Base de datos relacional y motor de reglas.	Transacciones, restricciones, funciones y disparadores.	Integridad garantizada por el motor.	Columnas creadas a mano desde el editor de tablas generaron migraciones correctivas (0026–0027).
PostGIS	Búsqueda por distancia.	Índices espaciales para «cerca mío».	Búsqueda en una sola consulta.	—
pg_cron	Tareas programadas.	Corre dentro de la base, sin servidor de tareas.	Cierre automático cada minuto.	Un error dentro del bucle abortaba todo el job (se corrigió en la 0029).

6.5 Servicios externos

Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
Google Maps Platform	Mapas y geocodificación web.	Cobertura y calidad en Argentina.	Mapa familiar para el usuario.	Las claves deben restringirse por paquete y dominio para evitar consumo de terceros.
OSRM (servidor demo)	Ruta hasta la cochera.	Gratis y sin clave.	Sin costo ni registro.	Sin garantía de disponibilidad; envía ubicaciones a un tercero; exige atribución a OpenStreetMap.

6.6 Herramientas de desarrollo, pruebas y control de versiones

Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
Jest 29 + **jest-expo**	Tests unitarios.	Preset oficial de Expo.	111 casos en 12 archivos sobre reglas puras (precio, disponibilidad, horas, validaciones, documentos de titularidad, huella de imágenes).	No se cubren componentes de interfaz.

Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
pg (node-postgres) 8	Pruebas de concurrencia contra la base real.	Conexiones simultáneas reales.	Detecta carreras que un test secuencial no ve.	Debe usarse con usuarios de prueba.
Git + GitHub	Control de versiones.	Estándar de la industria.	Ramas por funcionalidad; historial descriptivo.	Trabajo sin commitear acumulado en algunas etapas.
GitHub Actions	Integración continua.	Recomendación de la auditoría.	Tipos, tests y zonas horarias en cada push.	No ejecuta las pruebas SQL (necesitan la base).
Supabase SQL Editor	Aplicar migraciones y pruebas SQL.	Acceso directo sin CLI.	Simple.	Ejecución manual: orden y aplicación dependen de la disciplina del equipo.
Figma	Diseño de pantallas.	Colaboración visual.	Fuente de verdad del rediseño.	Algunos colores del diseño no cumplían contraste AA.
Claude Code (asistente de IA)	Asistencia en programación, revisión y documentación.	Acelerar el desarrollo y la documentación.	Explicaciones de cada decisión dentro del código y las migraciones.	Requiere revisión humana de cada cambio y pruebas reales (las migraciones las corrió y verificó el equipo).
Codex (asistente de IA de OpenAI)	Auditoría técnica del sistema completo: errores, seguridad, rendimiento y arquitectura.	Tener una revisión independiente de la del equipo antes de la entrega.	Encontró fallas que las pruebas manuales no mostraban (reservas manipulables, dobles reembolsos, calificaciones sobre cocheras ajenas). El equipo realizó todas las correcciones (ver 9.6).	Cada hallazgo hubo que confirmarlo y corregirlo con migraciones y pruebas contra la base real.
Visual Studio Code, **Node.js** 22, **npm**	Edición y ejecución.	Estándar.	—	El SDK 57 exige Node 22.13 o superior.

6.7 Despliegue y distribución

Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
EAS Build (Expo)	APK de Android (perfiles *development* y *preview*).	Compilar en la nube.	APK instalable por enlace.	Empaqueta desde Git: un cambio sin commitear no viaja al build; no lee el `.env` local.

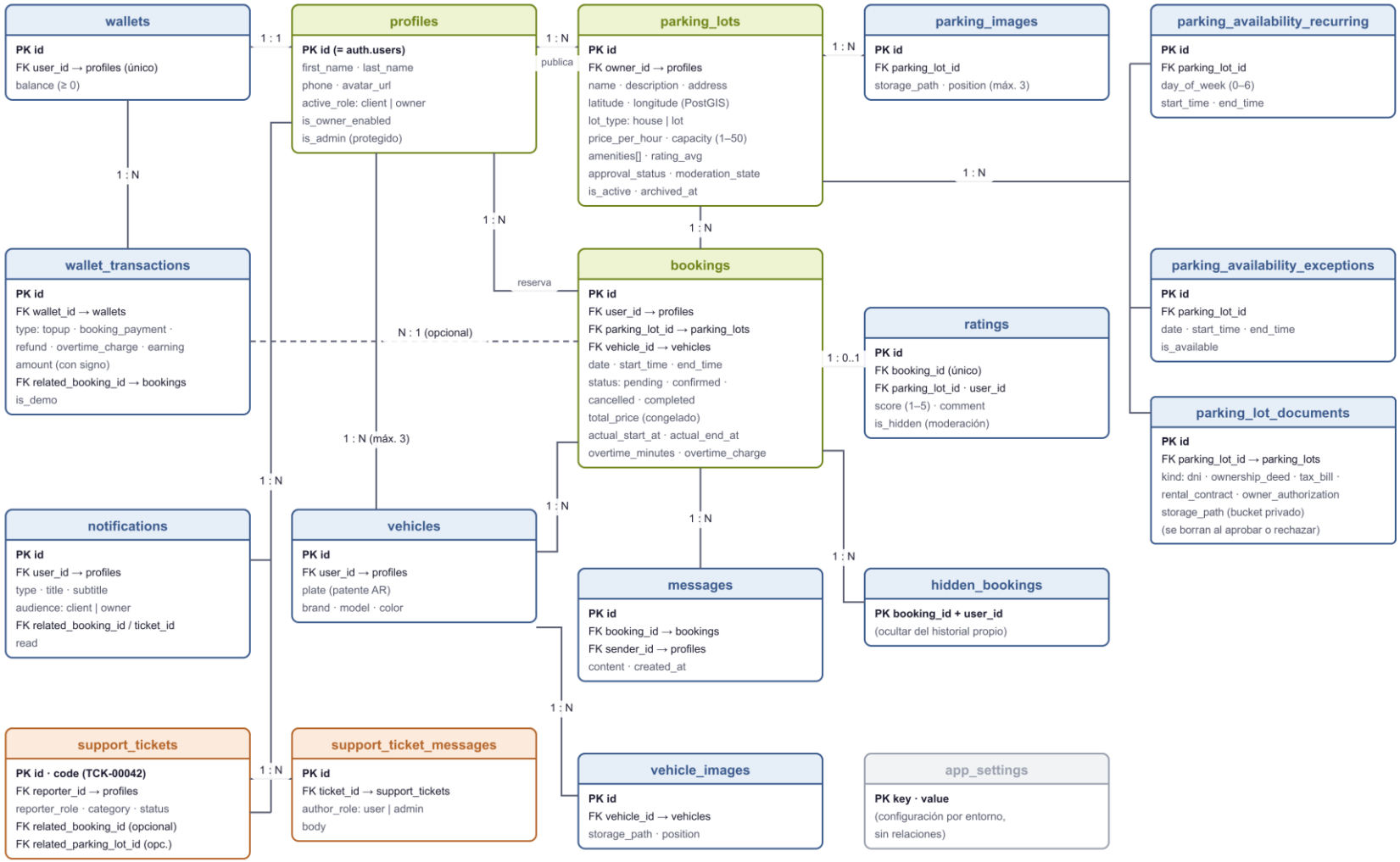
Tecnología	Finalidad	Motivo de elección	Ventajas	Dificultades / limitaciones
Cloudflare Workers (Wrangler)	Hosting de la app web y del panel.	Gratuito y simple para sitios estáticos.	Publicación con un comando.	Hace falta `not_found_handling: single-page-application`; un archivo `_redirects` se rechaza por bucle infinito.

7. Modelo de datos

7.1 Diagrama entidad-relación

El estado completo del sistema vive en una base PostgreSQL: 18 tablas y 1 vista, construidas ejecutando en orden las 83 migraciones de `supabase/migrations/`.

7.1 Diagrama entidad-relación



PK = clave primaria · FK = clave foránea. Las líneas punteadas son relaciones opcionales. Vista adicional: public_profiles (solo nombre y foto).

Figura 3. Modelo entidad-relación (tablas del esquema público; `auth.users` lo administra Supabase).

7.2 Principales entidades

Tabla	Propósito	Datos principales
`profiles`	Datos del usuario; extiende `auth.users`. Se crea al registrarse.	Nombre, teléfono, foto, rol activo, si habilitó el modo propietario, si es administrador.
`wallets`	Billetera, una por usuario.	Saldo (nunca negativo).
`wallet_transactions`	Registro contable de cada movimiento.	Tipo (carga, pago, reembolso, excedente, cobro), importe con signo, reserva asociada, marca `is_demo`.
`parking_lots`	La publicación: centro del dominio.	Ubicación, tipo, precio por hora, capacidad, servicios, promedio de calificación; tres dimensiones de estado: revisión (`approval_status`), moderación (`moderation_state`) y actividad (`is_active`), más archived_at; relación de quien publica (holder_role) y sello de titularidad (ownership_verified_at / _by).
`parking_images`	Fotos de la cochera (máx. 3).	Ruta en Storage y orden.
parking_lot_documents	Documentos de titularidad de la cochera.	Tipo (DNI, escritura, boleta de impuesto/ABL, contrato, autorización, habilitación) y ruta en un *bucket* privado; uno por tipo; se borran al aprobar o rechazar.
`parking_availability_recurring`	Horarios habituales por día de la semana.	Día, hora de inicio y fin.
`parking_availability_exceptions`	Días puntuales bloqueados.	Fecha, horario, disponible o no.
`vehicles` / `vehicle_images`	Vehículos del conductor (máx. 3) y sus fotos.	Patente, marca, modelo, color; fotos en un *bucket* privado.
`bookings`	La reserva: operación central.	Quién, dónde, con qué vehículo; fecha y horario pactado; estado; precio congelado; inicio y fin reales; excedente cobrado; marcas de avisos enviados.
`ratings`	Calificación de una reserva.	Puntaje 1–5, comentario, marcas de moderación.
`messages`	Chat; cada mensaje pertenece a una reserva y se agrupa por cochera.	Remitente, contenido, fecha.
`hidden_bookings`	Reservas que un usuario quitó de su historial.	Usuario y reserva.
`notifications`	Avisos dentro de la app.	Tipo, título, subtítulo, audiencia (conductor o propietario), reserva o ticket relacionado, leída.

Tabla	Propósito	Datos principales
`support_tickets` / `support_ticket_messages`	Soporte con el equipo.	Código `TCK-00042`, rol desde el que se abrió, categoría, estado, vínculo opcional; mensajes del usuario y del equipo.
`app_settings`	Configuración por entorno.	Por ejemplo, `demo_topup_enabled`.
`public_profiles` (vista)	Datos públicos de otros usuarios.	Solo nombre y foto (sin teléfono ni roles).

7.3 Relaciones entre los datos

Relación	Cardinalidad	Comportamiento ante el borrado
Usuario → billetera	1 : 1	Se crea junto con el perfil.
Billetera → movimientos	1 : N	Los movimientos se conservan siempre.
Usuario (propietario) → cocheras	1 : N	La cochera con historial se archiva en lugar de borrarse, y se borran sus chats.
Cochera → fotos, horarios, excepciones	1 : N	Se borran con la cochera.
Cochera → documentos de titularidad	1 : N (uno por tipo)	Se borran con la cochera y también al aprobarla o rechazarla.
Usuario (conductor) → reservas	1 : N	—
Cochera → reservas	1 : N	No se puede borrar una cochera con reservas vigentes.
Vehículo → reservas	1 : N	La reserva conserva su historial.
Reserva → calificación	1 : 0..1	Unicidad por reserva.
Reserva → mensajes, ocultamientos	1 : N	Dependen de la reserva.
Reserva → movimientos	1 : N (opcional)	Al borrar una reserva el movimiento queda con la referencia en nulo: **el historial contable no desaparece**.
Usuario → notificaciones y tickets	1 : N	—
Ticket → mensajes	1 : N	Dependen del ticket.

7.4 Restricciones y validaciones importantes

Restricción	Regla que garantiza
`balance >= 0` en `wallets`	La billetera nunca queda negativa.
`score` entre 1 y 5; unicidad de `booking_id` en `ratings`	Puntaje válido y una calificación por reserva.
`capacity` entre 1 y 50	Capacidad razonable de la cochera.
Lista cerrada de `amenities` y `lot_type`	Solo servicios y tipos conocidos.

Restricción	Regla que garantiza
Índice único día + horario por cochera	Sin franjas duplicadas.
Índice único de movimiento por reserva y tipo	Nunca dos reembolsos, cobros o excedentes de la misma reserva.
Disparadores de conteo	Máximo 3 vehículos por usuario y 3 fotos por cochera.
`brand` y `model` obligatorios (NOT VALID)	Todo vehículo nuevo o editado tiene marca y modelo, sin romper los cargados antes.
Límites de Storage	Archivos de hasta 5 MB y solo jpeg, png o webp.
Al menos una foto para publicar o aprobar	Ninguna cochera sin fotos llega a la búsqueda (photo_required / photos_missing).
Un documento por tipo y cochera; lista cerrada de tipos	Documentación de titularidad ordenada y sin duplicados.
Disparadores de re-revisión	Una cochera aprobada que cambia datos, tarifa, servicios o fotos vuelve a pendiente.
Validaciones dentro de las RPC	Disponibilidad, capacidad, margen, horario no pasado, duración, vehículo libre, saldo, ventana de inicio, plazo de cancelación.

7.5 Criterios para proteger la integridad de la información

- **Escritura solo por funciones:** desde la migración 0046 el cliente solo puede *leer* `bookings`; crear, iniciar, finalizar y cancelar pasa por funciones que validan todo.
- **Transacciones atómicas** en toda operación que toca dinero.
- **Bloqueos y condiciones de estado** para que dos operaciones simultáneas no liquiden dos veces (0047, 0048, 0067).
- **Precio congelado:** un cambio de tarifa no altera reservas existentes.
- **Se registra lo cobrado, no lo teórico:** si el saldo no alcanzaba para el excedente, queda registrado el monto real.
- **Ocultar no es borrar:** reseñas ocultas, reservas quitadas del historial y cocheras archivadas conservan sus datos.
- **Columnas sensibles protegidas por disparadores:** `is_admin`, estado de moderación y campos de la revisión y el sello de titularidad.
- **Datos personales por el tiempo justo: los documentos de titularidad se borran al decidir la revisión (0079).**
- **Datos del servidor, no del cliente:** la calificación toma la cochera de la reserva, no la que envía la app (0049).
- **Migraciones como única vía de cambio del esquema**, versionadas e idempotentes.

7.6 Ciclo de vida de una reserva

La reserva es una máquina de estados con transiciones controladas por funciones del servidor. El estado *pendiente de pago* existe en la base y bloquea el horario, reservado para la futura integración con Mercado Pago.

7.6 Ciclo de vida de una reserva

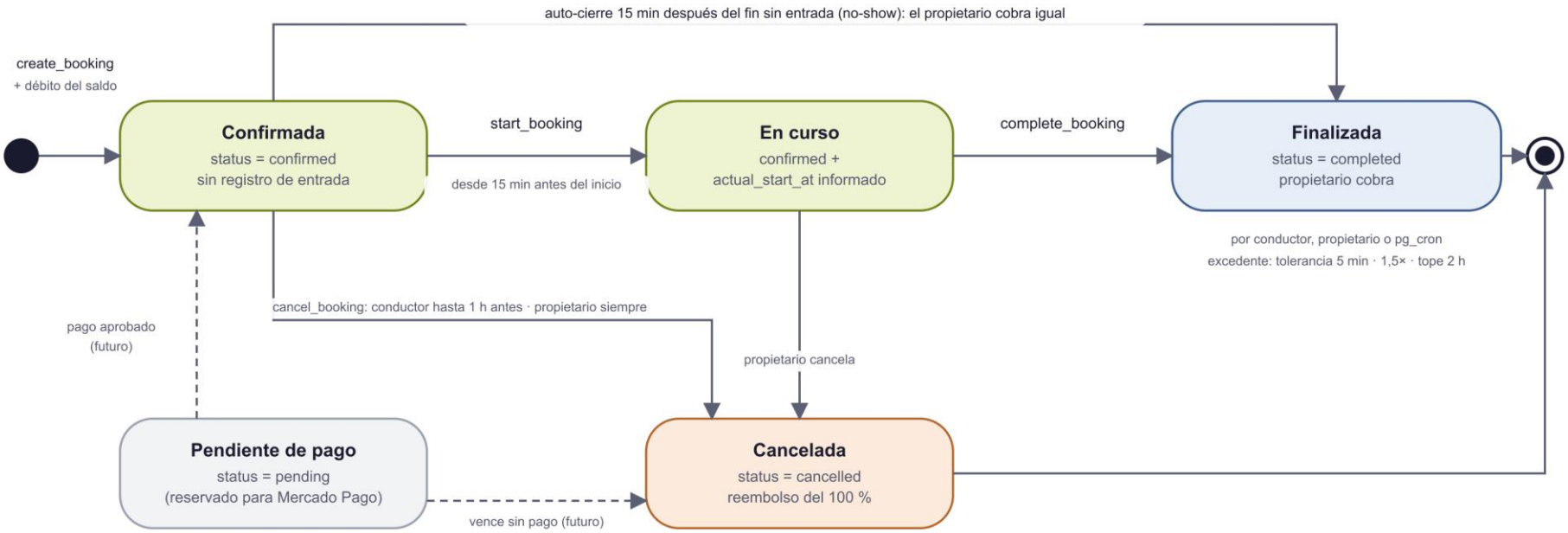


Figura 4. Estados de una reserva y funciones que producen cada transición.

7.7 Funciones principales de la base

Función	Uso	Qué hace
`create_booking`	App	Valida y crea la reserva, debita y notifica, en una transacción.
`start_booking` / `complete_booking` / `cancel_booking`	App	Transiciones de la reserva con sus reglas.
`_finalize_booking` / `_assert_slot_bookable`	Internas	Liquidación del cierre y validación de disponibilidad (reutilizables por Mercado Pago).
`search_parking_lots` / `parking_lots_available_now`	App	Búsqueda espacial con filtros y estado «disponible ahora».
`list_*` / `get_*`	App	Listados paginados por cursor y detalles armados en una sola consulta.
`delete_parking_lot` / `create_support_ticket` / `topup_wallet`	App	Borrar o archivar (cerrando los chats), crear ticket con su mensaje, recarga simulada.
`auto_close_stale_bookings`, `send_booking_end_reminders`, `send_booking_day_before_reminders`	pg_cron	Cierre automático y avisos.
lot_missing_documents y disparadores de titularidad y re-revisión	Internas	Calculan qué documentos faltan, impiden publicar o aprobar sin fotos ni documentos y devuelven a revisión una cochera aprobada que se edita.
`admin_*` (moderación, métricas, usuarios, transacciones, reseñas, tickets)	Panel	Verifican `is_admin` y devuelven exactamente lo que el panel necesita.

8. Diseño de interfaces

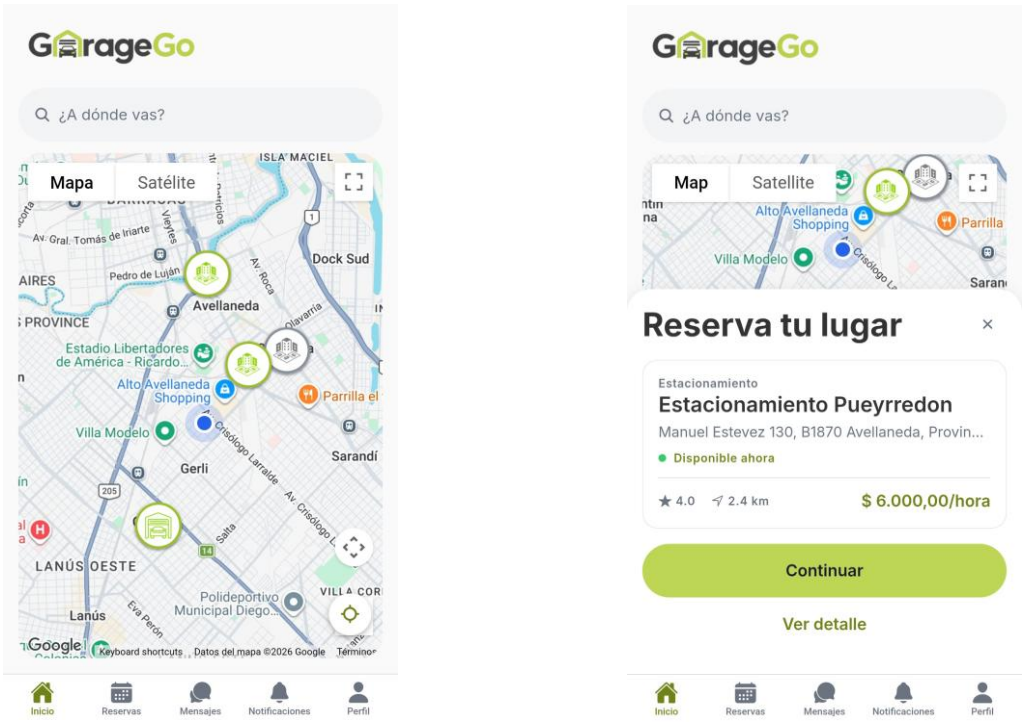
8.1 Principales pantallas del sistema

La app tiene **66 archivos de rutas** repartidos en tres grupos. Las pantallas principales:

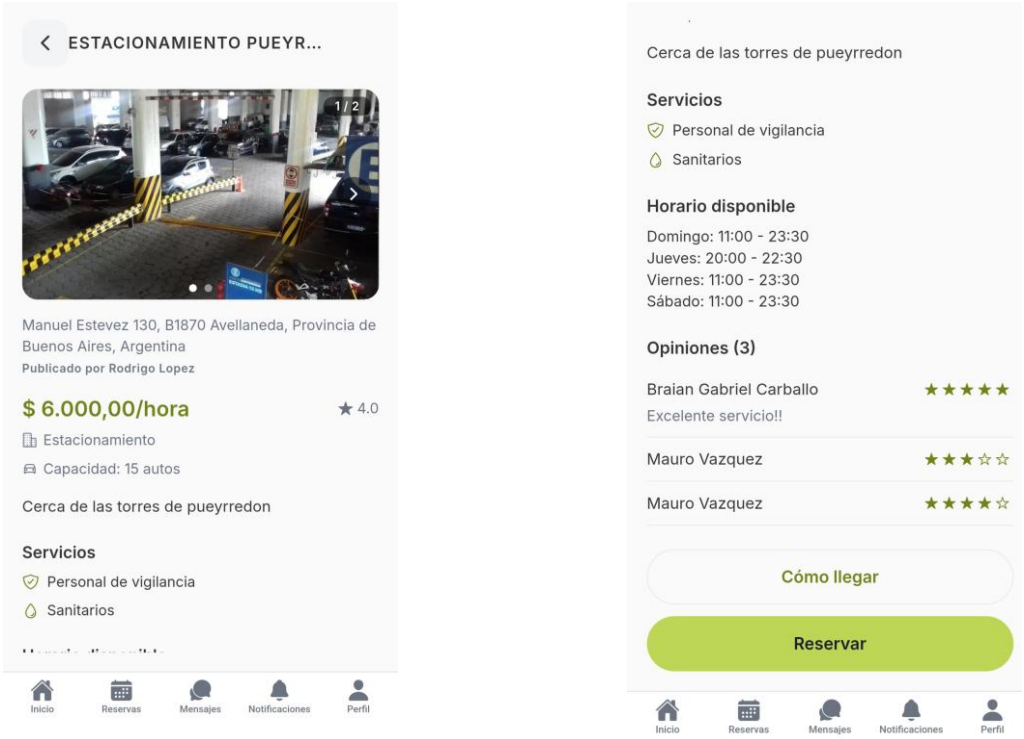
Pantalla	Rol	Propósito
Iniciar sesión / Registrarse / Recuperar contraseña	Visitante	Acceso a la cuenta; recuperación en tres pasos con código.
Inicio (mapa)	Conductor	Mapa con todos los estacionamientos publicados, pines por tipo y disponibilidad, buscador por dirección y vista previa al tocar un pin.
Lista de resultados	Conductor	Resultados con filtros de distancia, precio y calificación.
Ficha de la cochera	Conductor	Galería, precio, capacidad, servicios, horarios, reseñas, «Cómo llegar» y «Reservar».
Reserva → Pago → Comprobante	Conductor	Fecha, horario y vehículo; pago con saldo; comprobante.
Mis reservas / Detalle / Historial	Conductor	Cuenta regresiva, sesión activa con anillo de progreso, calificar.
Mensajes / Chat	Ambos	Una conversación por cochera, en tiempo real.
Notificaciones	Ambos	Avisos del rol activo; borrar uno o todos; abrir la pantalla relacionada.
Perfil, Ajustes, Billetera, Vehículos, Centro de ayuda, Legal, Información	Ambos	Cuenta, saldo, soporte y textos legales.
Mis publicaciones / Asistente de 5 pasos	Propietario	Publicaciones por estado; alta y edición, con titularidad.
Reservas recibidas / Detalle	Propietario	Conductor, vehículo, dirección; finalizar o cancelar.
Dashboard / Mis estadísticas	Propietario	Actividad del día, ingresos y ocupación por cochera.
Panel: Dashboard, Inventario, Usuarios, Soporte	Administrador	Métricas, moderación con documentos de titularidad, directorio y reseñas, tickets y transacciones.

Capturas

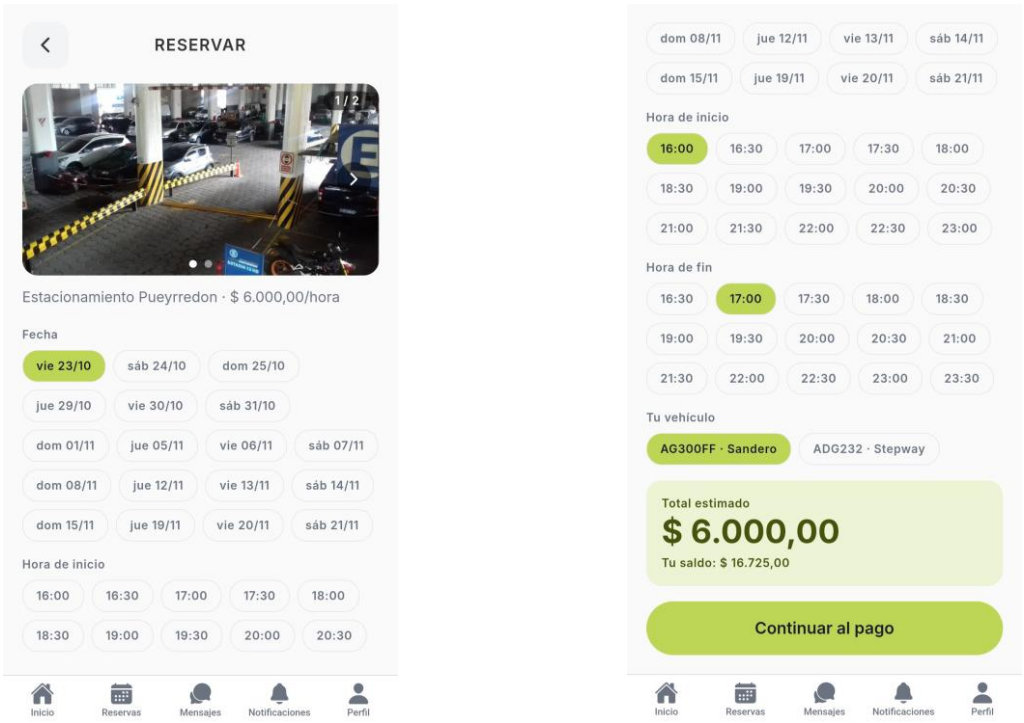
Capturas de la versión publicada: la app en un celular, con el rol de conductor y de propietario, y el panel de administración en una computadora. Se tomaron con cuentas y datos de prueba.



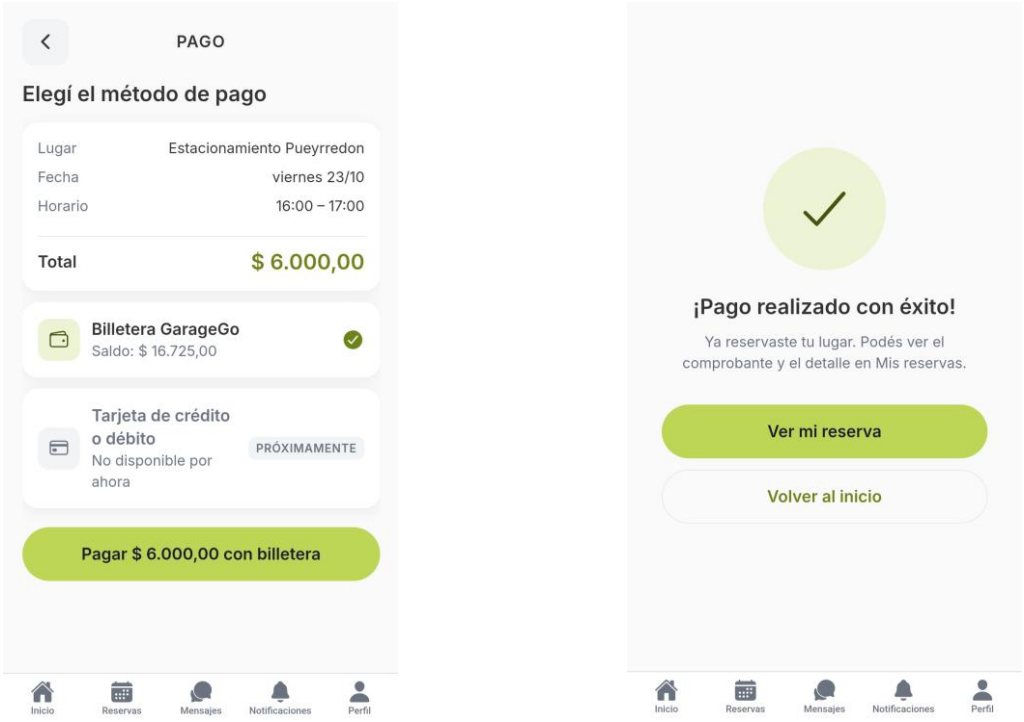
Captura 1. Inicio del conductor: mapa con los estacionamientos publicados y vista previa al tocar un pin.



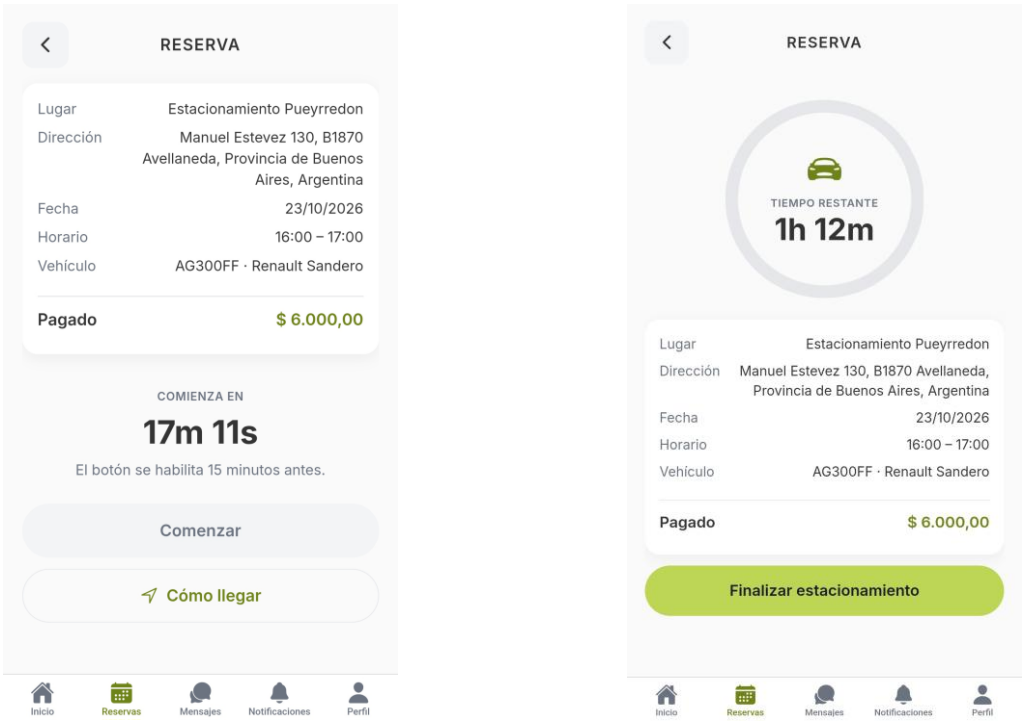
Captura 2. Ficha de la cochera: galería, precio, tipo, capacidad, servicios, horarios y opiniones.



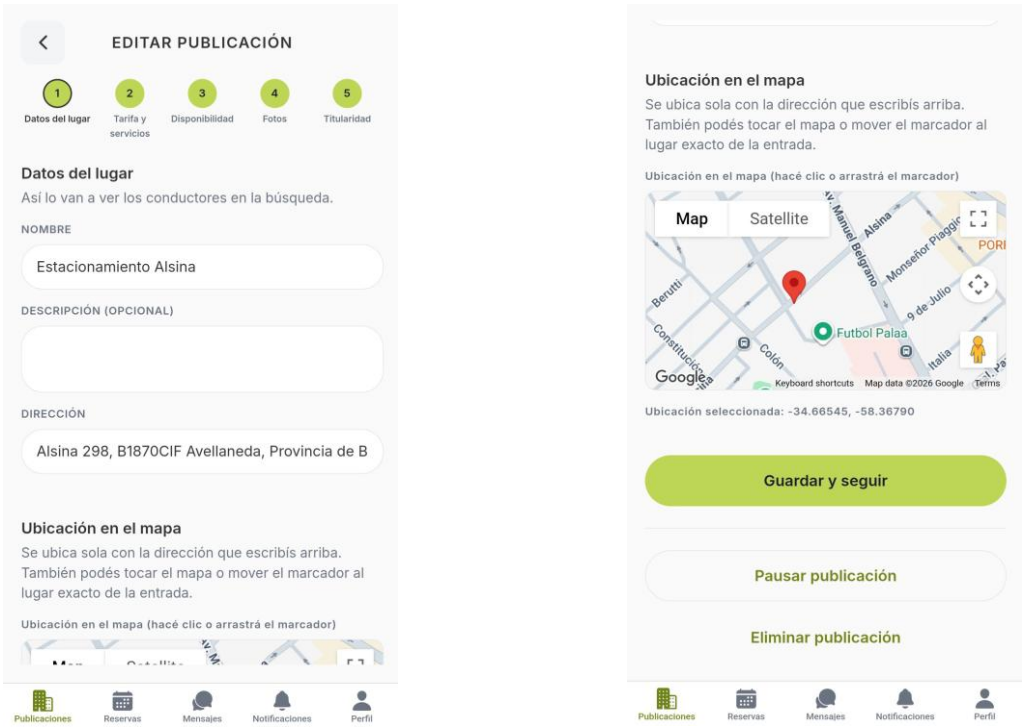
Captura 3. Reserva: elección de fecha, horario de inicio y fin y vehículo, con el total estimado y el saldo.



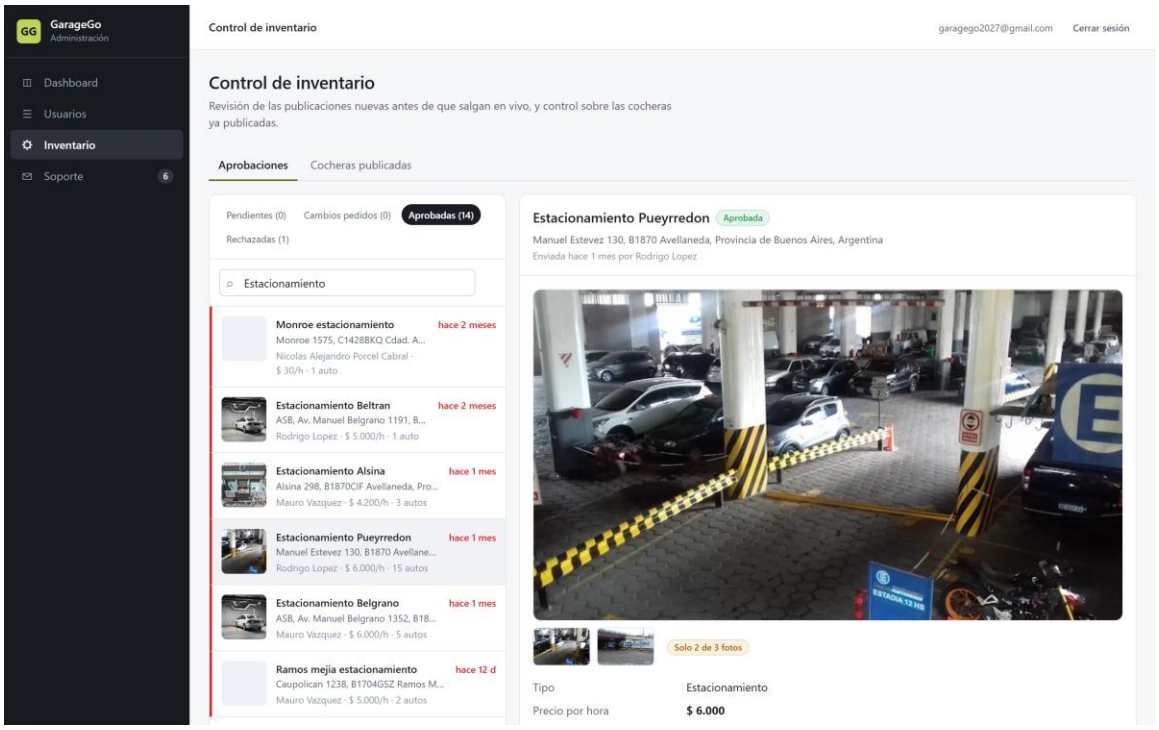
Captura 4. Pago con la billetera de GarageGo y confirmación de la reserva.



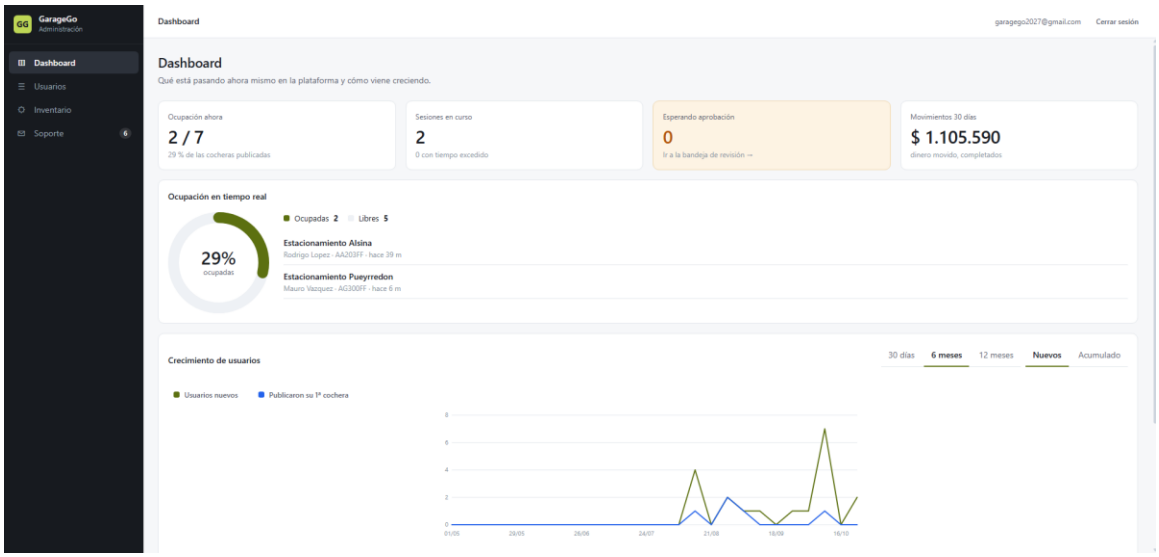
Captura 5. Detalle de la reserva con la cuenta regresiva y sesión en curso con el tiempo restante.



Captura 6. Propietario: asistente de publicación de 5 pasos (datos del lugar y ubicación en el mapa).



Captura 7. Panel de administración: control de inventario y revisión de publicaciones.



Captura 8. Panel de administración: dashboard con ocupación en tiempo real y crecimiento de usuarios.

8.2 Flujo de navegación

La guardia de navegación de `app/_layout.tsx` decide el grupo de pantallas: sin sesión, `(auth)`; con sesión, `(client)` u `(owner)` según el rol activo. **Cambiar de rol reemplaza la navegación completa**: el usuario nunca ve opciones que no le corresponden. La guardia no redirige hasta haber cargado el perfil, para que un propietario no vea por un instante la pantalla de conductor.

8.2 Flujo de navegación

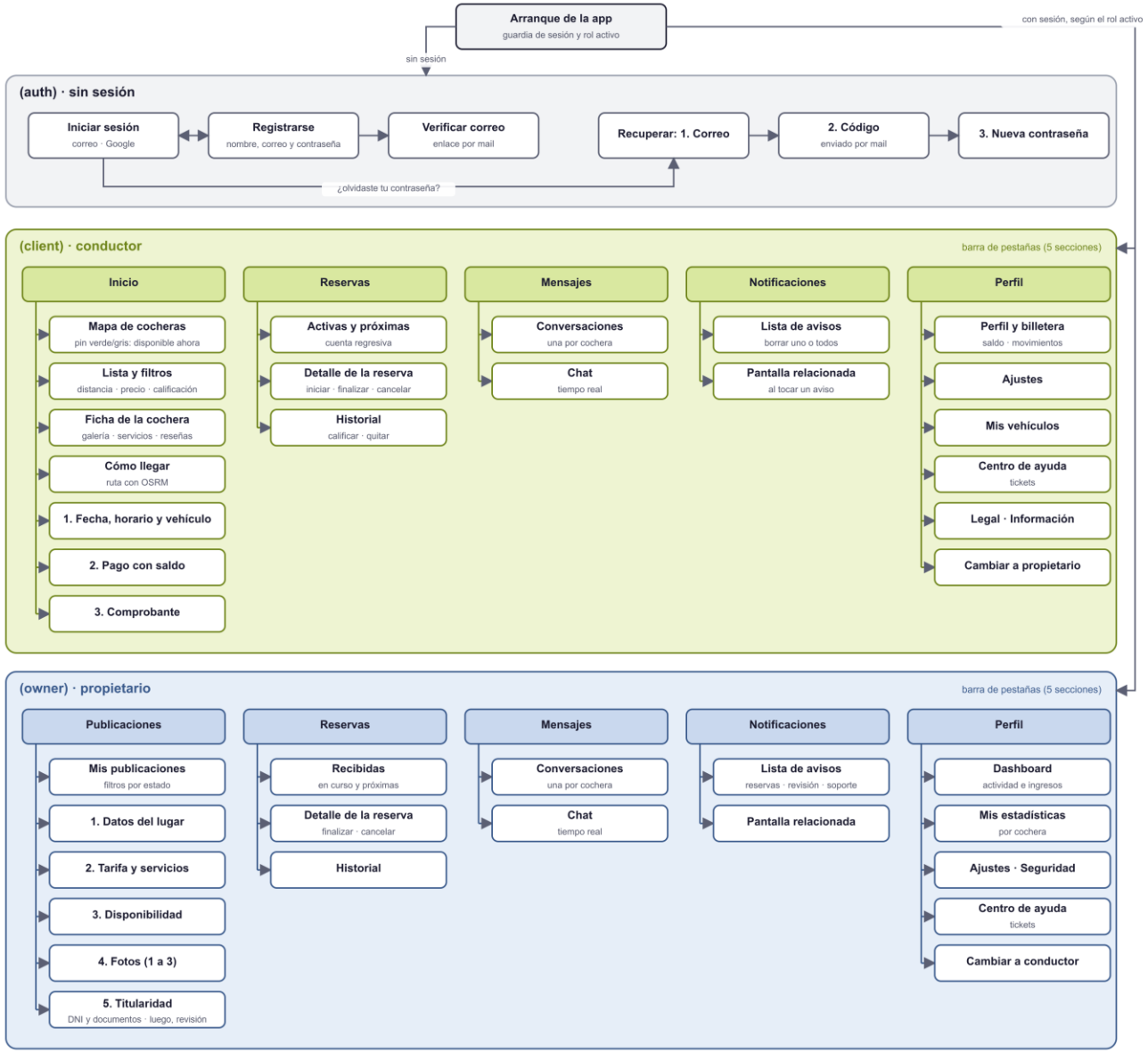


Figura 5. Mapa de navegación de la app por grupo de rutas y pestaña.

8.3 Criterios de experiencia de usuario

- **Identidad visual:** verde lima de marca (`#BDD655`) sobre fondos claros y tipografía Inter; el color señala la acción principal de cada pantalla.
- **Sistema de diseño:** todos los valores visuales salen de `src/theme/` (colores, 6 estilos tipográficos, escala de espaciado de 6 pasos, radios, sombras). No hay colores ni medidas sueltas en las pantallas.
- **Cuatro estados por pantalla:** cargando, vacío (con la acción que corresponde), error (con reintento) y con datos.
- **Confirmación de acciones irreversibles** (cancelar, eliminar, borrar) con un diálogo que enuncia la consecuencia.
- **Lo activo separado de lo pasado:** reservas vigentes e historial en pantallas distintas.
- **Información de un vistazo:** cuenta regresiva con días (`2d 20h 06m`), anillo de progreso, pines verdes o grises según disponibilidad ahora.
- **Mapa y lista sobre la misma búsqueda**, sin perder filtros.
- **Tareas largas en pasos:** el asistente de publicación divide la carga en 5 pantallas; la recuperación de contraseña en 3.
- **Detalles de uso real:** campo del chat pegado al teclado, dirección escrita que se ubica sola en el mapa, ver u ocultar contraseña, botón «volver» coherente, la fuente Inter se carga sin bloquear el arranque.

8.4 Validaciones de los formularios

Cada formulario valida en el cliente con esquemas `zod` (para avisar al instante) y el servidor vuelve a validar lo crítico (para que nada dependa de la app).

Formulario	Campo	Regla
Registro	Nombre, apellido	Obligatorios.
Registro	Correo	Obligatorio y con formato válido.
Registro	Contraseña	Mínimo 8 caracteres, al menos una mayúscula, una minúscula y un número; confirmación igual.
Recuperación	Código	Numérico, de 6 a 10 dígitos.
Recuperación	Nueva contraseña	Mínimo 8 caracteres y confirmación igual.
Perfil	Teléfono	Opcional; si se completa, teléfono argentino válido (acepta +54, 9, 0, 15 y separadores).
Vehículo	Patente	Formato argentino: viejo `ABC123` o Mercosur `AB123CD`.
Vehículo	Marca, modelo, color	Marca y modelo obligatorios (máx. 40); color opcional (máx. 30).
Publicación · paso 1	Nombre, dirección, ubicación	Obligatorios; latitud entre −90 y 90, longitud entre −180 y 180.
Publicación · paso 2	Precio, capacidad, tipo, servicios	Precio mayor a 0; capacidad entera de 1 a 50; tipo y servicios de listas cerradas.

Formulario	Campo	Regla
Publicación · paso 3	Disponibilidad	Fin posterior al inicio; sin franjas duplicadas para el mismo día.
Publicación · paso 4	Fotos	De 1 a 3, distintas entre sí (se compara una huella de cada imagen); máximo 5 MB; jpeg, png o webp.
Publicación · paso 5	Titularidad	Relación con el lugar obligatoria; DNI y documentos según esa relación, como imágenes jpeg, png o webp de hasta 5 MB.
Reserva	Fecha, horario, vehículo	Futuro (hora argentina); mínimo 30 minutos en tramos de 30; dentro de la disponibilidad y la capacidad; vehículo propio y libre.
Recarga de saldo	Importe	Máximo $50.000 por recarga y saldo total máximo $200.000.
Ticket	Categoría y descripción	Categoría obligatoria; vínculo opcional pero siempre propio.

8.5 Tratamiento de errores y mensajes al usuario

Los errores del servidor se traducen a lenguaje comprensible en el `api.ts` de cada dominio. Ejemplos:

Código del servidor	Mensaje que ve el usuario
`insufficient_balance`	«No tenés saldo suficiente para esta reserva.» (con acceso a cargar saldo)
`buffer_conflict` / `overlap_conflict`	«Ese horario ya no está disponible. Elegí otro.»
`cancel_too_late`	Aviso de que el conductor solo puede cancelar hasta 1 hora antes.
`vehicle_required` / `vehicle_busy`	«Elegí un vehículo para esta reserva.» / el vehículo ya tiene otra reserva en ese horario.
`start_too_early` / `start_window_over`	A partir de qué hora puede iniciar / el horario reservado venció.
`invalid_duration`	«La reserva tiene que durar al menos 30 minutos, en tramos de 30.»
`changes_not_made`	Qué campos pedidos por el equipo siguen sin corregir.
photo_required	«Subí al menos una foto del lugar para publicarlo.»
ownership_docs_required	«Faltan los documentos que acreditan que sos el dueño o que tenés permiso para publicar este lugar.»

Como red final, `src/lib/errorMessages.ts` impide que lleguen a pantalla mensajes internos de Postgres (nombres de tablas, columnas o restricciones) y los reemplaza por un mensaje genérico. Los errores se muestran en un `Banner` con posibilidad de reintentar; las acciones que pueden fallar por reglas de negocio advierten antes (por ejemplo, el diálogo de cancelación).

8.6 Accesibilidad

La decisión más importante del sistema de diseño: el verde lima de la marca tiene un contraste de ~1,6:1 contra blanco, muy por debajo del mínimo AA (4,5:1). Se separó en dos

tokens: `primary` (`#BDD655`) para **rellenos**, con texto oscuro encima (`onPrimary`, 10,1:1), y `accent` (`#6E8419`) para **texto e íconos** de marca sobre fondo claro (5,5:1). El gris secundario se fijó en el tono que alcanza 5,3:1. La decisión está documentada en `src/theme/colors.ts`.

- Botones con forma de píldora y áreas táctiles amplias.
- Textos e íconos con contraste AA.
- Mensajes de error en texto, no solo en color.
- **Pendiente:** etiquetas para lectores de pantalla en íconos y controles, prueba con tamaños de fuente ampliados y modo oscuro (la paleta ya está definida).

9. Seguridad

> Principio rector: **nada de lo que la app haga puede otorgar un permiso.** La app se distribuye a los usuarios y cualquiera puede leer sus claves y llamar al backend directamente. Por eso ningún control de seguridad vive en la app: ocultar un botón es una decisión de interfaz, no de seguridad.

9.1 Registro, autenticación y autorización

Aspecto	Implementación
Registro	Supabase Auth con verificación de correo; un disparador crea perfil y billetera.
Autenticación	Correo y contraseña, o Google con OAuth + **PKCE** en el navegador del sistema (la app nunca ve la contraseña de Google y otra app no puede interceptar el código).
Recuperación	Código numérico por correo; respuesta idéntica exista o no la cuenta.
Autorización	Políticas RLS por fila en todas las tablas + funciones que validan quién las llama + disparadores que protegen columnas sensibles.
Panel	Solo correo y contraseña; las cuentas administradoras se crean desde la consola de Supabase y `is_admin` se marca desde el SQL Editor.

9.2 Roles y permisos

Recurso	Conductor	Propietario	Administrador
Perfil propio	Lee y edita	Lee y edita	Lee todos
Perfiles ajenos	Solo nombre y foto (vista)	Solo nombre y foto (vista)	Lee todos
Billetera y movimientos	Solo los propios	Solo los propios	Resumen y listado por función
Cocheras	Aprobadas y activas; las reservadas aunque se pausen	Las propias (crear, editar, pausar, archivar)	Todas; modera por función
Reservas	Las propias (solo lectura directa; escritura por funciones)	Las de sus cocheras (lectura; finalizar y cancelar por funciones)	Agregados por función
Vehículos	Los propios (fotos privadas)	Solo el de una reserva suya	Por función (ficha de usuario)
Mensajes	Hilos donde es parte	Hilos donde es parte	—
Calificaciones	Crea sobre reservas propias finalizadas con entrada	Lee las de sus cocheras	Lee todas; oculta
Tickets	Los propios	Los propios	Todos; responde y resuelve
Documentos de titularidad	—	Sube y ve los de sus cocheras; los borra mientras no estén aprobadas	Los lee durante la revisión y los borra al decidir

Recurso	Conductor	Propietario	Administrador
`is_admin`, estado de moderación	No puede modificarlos	No puede modificarlos	Solo vía consola / funciones

9.3 Protección de contraseñas y datos sensibles

- GarageGo **nunca ve ni guarda contraseñas**: las administra Supabase Auth (almacenadas con hash).
- La política de contraseñas para cuentas nuevas exige 8 caracteres con mayúscula, minúscula y número.
- Solo la **anon key** viaja en la app y el panel; la `service_role` nunca se usa en el código (`admin-panel/.env.example` lo advierte). Los `.env` están fuera del control de versiones.
- Desde la 0051 cada usuario lee solo su propio perfil; el resto ve nombre y foto a través de la vista `public_profiles`. El teléfono y los roles dejaron de estar expuestos.
- Los documentos de titularidad (DNI, escrituras, contratos) y las fotos de los vehículos (suelen mostrar la patente) se guardan en *buckets* privados: solo los ven su dueño y el equipo, con URLs firmadas que vencen (0077, 0078). Los documentos se borran al aprobar o rechazar la publicación (0079).
- Las claves de Google Maps son visibles por naturaleza (cliente); la protección es restringirlas por paquete y firma (Android) y por dominio (web).

9.4 Validación de las entradas

- **En el cliente:** esquemas `zod` en todos los formularios (sección 8.4) y validación de tamaño y tipo de imagen antes de subir.
- **En el servidor:** restricciones de tabla, funciones que validan cada parámetro (estado, horarios, propiedad del vehículo, saldo, capacidad) y límites de Storage.
- **Sin SQL dinámico desde el cliente:** PostgREST parametriza las consultas; las funciones no concatenan texto del usuario.
- **Datos que el cliente no decide:** disparadores que ignoran lo que envía la app en columnas como la cochera de una calificación o los campos de moderación.

9.5 Manejo de sesiones y tokens

- Supabase emite un **JWT** de acceso con vencimiento y un token de renovación; la app los renueva automáticamente.
- La sesión se guarda en `AsyncStorage`, aislado por el sistema operativo.
- El arranque escucha los cambios de sesión: un cierre o vencimiento se refleja al instante en toda la app; las cargas de un usuario anterior se descartan al cambiar de cuenta.
- Cerrar sesión elimina la sesión local y redirige a `(auth)`.

9.6 Riesgos identificados y controles implementados

Se realizó una auditoría técnica integral del sistema con Codex (errores, seguridad, rendimiento y arquitectura). Todos los hallazgos se corrigieron y se verificaron contra la base real.

Severidad	Hallazgo	Control implementado	Migración
Crítica	Se podían insertar o modificar reservas directamente (reserva sin pagar, inflar el precio y cobrar un reembolso mayor).	El cliente solo lee `bookings`; escritura únicamente por funciones; privilegios revocados; `confirm_booking` deja de ser invocable.	0046
Crítica	Se podía calificar una cochera ajena usando una reserva propia.	Disparador que toma la cochera de la reserva e ignora lo enviado.	0049
Alta	Cancelar o finalizar dos veces a la vez duplicaba reembolsos o cobros.	Bloqueo de la fila, actualización condicionada al estado e índice único contable.	0046–0047
Alta	La disponibilidad y la moderación se validaban solo en la app.	`_assert_slot_bookable` replica las reglas en el servidor con *lock* por cochera.	0048
Alta	Todos los usuarios podían leer teléfonos, roles y quiénes eran administradores.	Vista pública con nombre y foto; `profiles` solo propio.	0050–0051
Alta	Funciones invocables sin sesión y una que revelaba el historial de lugares de cualquiera.	Revocación de `EXECUTE` a `public` y `anon`; función eliminada.	0052
Alta	Borrar una cochera eliminaba reservas pagadas sin reembolso.	Bloqueo con reservas vigentes y archivado con historial.	0053
Media	Recarga simulada ilimitada y sin distinguir de pagos reales.	Topes de saldo, marca `is_demo` y bandera por entorno.	0057, 0061, 0074
Media	Archivos de cualquier tamaño y tipo en los buckets.	5 MB y solo jpeg, png o webp.	0058
Media	Las fotos de los vehículos estaban en un *bucket* público: con la URL se veía la patente sin iniciar sesión.	*Bucket* privado y URLs firmadas.	0077
—	Auto-asignarse administrador o auto-aprobar una publicación.	Disparadores `protect_profile_is_admin` y `protect_parking_lot_moderation`.	0033, 0037
—	Recursión infinita entre políticas de reservas y cocheras.	Funciones auxiliares `security definer` que rompen el ciclo.	0040
—	Cualquiera podía publicar un lugar ajeno, y editar una publicación aprobada no pasaba por revisión.	Verificación de titularidad con documentos privados que se borran al decidir; re-revisión al editar; al menos una foto.	0078–0082

9.7 Aspectos de seguridad pendientes

#	Pendiente	Mitigación sugerida
1	Sesión guardada sin cifrar (`AsyncStorage`).	Migrar a `expo-secure-store` (ya está en la configuración).

#	Pendiente	Mitigación sugerida
2	Sin registro de auditoría de acciones administrativas.	Tabla de auditoría unificada (hoy hay trazas parciales: `reviewed_by`, `hidden_by`).
3	Sin baja de cuenta ni política de datos personales.	Implementar la eliminación (el botón ya existe) y definir retención de datos (Ley 25.326).
4	OSRM recibe ubicaciones del usuario y de la cochera.	Servidor de rutas propio o proveedor con acuerdo de datos.
5	Sin límite de intentos propio.	Depende de los límites de Supabase; evaluar reglas adicionales.
6	Textos legales en borrador.	Revisión legal antes de abrir al público.
7	Los documentos de titularidad se revisan a mano, sin validación automática de identidad.	Evaluar un servicio de validación de identidad antes de abrir al público.

10. Pruebas realizadas

10.1 Estrategia de prueba

Como las reglas críticas viven en la base de datos, la estrategia combina pruebas en cuatro niveles, cada una apuntando a un tipo de error distinto:

Nivel	Herramienta	Qué verifica	Volumen
Unitarias	Jest + jest-expo	Lógica pura del cliente: precio, horarios disponibles, capacidad, horas en Argentina, validaciones de formularios, dinero, paginación, saneamiento de errores, documentos de titularidad y huella de imágenes.	12 archivos · 111 casos
Zona horaria	`npm run test:tz`	Los mismos tests de fechas con el «teléfono» en 6 husos (Buenos Aires, UTC, Los Ángeles, Madrid, Tokio y Kiritimati, UTC+14): el resultado no debe cambiar.	6 ejecuciones
Integración en la base	Scripts SQL en `supabase/tests/`	Reglas y permisos contra las políticas reales: cada script arma sus datos, prueba y revierte todo; el reporte es una línea OK / FALLA por caso.	9 scripts · 76 casos
Concurrencia	`npm run test:concurrency` (node-postgres)	Carreras reales con conexiones simultáneas: la segunda operación debe esperar el *lock* y fallar sin duplicar dinero.	12 escenarios
Funcionales manuales	*Development build* en Android y versión web	Flujos completos de punta a punta por rol.	Ver 10.3
Integración continua	GitHub Actions	Verificación de tipos, tests y tests de zona horaria en app y panel en cada push.	Cada commit

10.2 Casos de prueba principales

Reglas de negocio y seguridad (scripts SQL)

ID	Caso	Resultado esperado	Obtenido
PA-01	Insertar una reserva directamente por la API.	Rechazado por permisos.	OK
PA-02	Modificar `total_price` de una reserva propia.	No cambia nada.	OK
PA-03	Calificar indicando una cochera ajena.	Se guarda con la cochera de la reserva.	OK
PA-04	Finalizar una reserva no iniciada.	`booking_not_started`.	OK
PA-05/06	Reservar un día bloqueado o fuera de horario.	`outside_availability`.	OK
PA-07	Reservar dentro del margen de 15 min de otra reserva.	`buffer_conflict`.	OK
PA-08	Borrar una cochera con reservas vigentes.	`lot_has_active_bookings`.	OK

ID	Caso	Resultado esperado	Obtenido
PA-09	Cancelar dos veces.	La segunda falla; un solo reembolso.	OK
PA-10	Borrar una cochera con historial.	Queda archivada.	OK
PA-11	Llamar `create_booking` sin sesión.	Sin permiso de ejecución.	OK
PA-12	Leer perfiles ajenos.	Solo el propio; la vista no expone teléfono.	OK
PL-01..09	Listados paginados, aislamiento entre usuarios, funciones del panel vedadas a usuarios comunes, Realtime del panel, bandera de recarga.	Cada usuario ve solo lo suyo.	OK (9/9)
PF-01..06	Detalle de reserva del propietario, tickets por rol, capacidad 2 (dos autos sí, tercero no; máximo simultáneo, no total), rango 1–50.	Según la regla.	OK
PJ-01..07	Servicios válidos, aviso previo único, no-show cobra el propietario, olvidada cobra 15 min, margen respetado, calificar solo con entrada, job cada minuto.	Según la regla.	**OK (7/7, 23/10/2026)**
PV-01..04	Mismo vehículo en dos cocheras (`vehicle_busy`), reservas consecutivas permitidas, iniciar con vehículo en uso (`vehicle_in_use`), recordatorio 24 h único.	Según la regla.	OK
PT-01..15	Titularidad: requisitos según la relación con el lugar, activar o aprobar sin documentos, documentos ajenos, lectura solo del dueño y del equipo, sello de verificación y borrado al decidir.	Según la regla.	OK (15/15, 16/10/2026)
PC-01..08	Eliminar una publicación (se archiva, se borran los chats, no se puede escribir ni abrir tickets) y re-revisión al editar (datos, tarifa y fotos sí; horarios no).	Según la regla.	OK (8/8, 23/10/2026)
PP-01..07	Al menos una foto para publicar y aprobar (photo_required, photos_missing) y cambio de dirección que quita el sello de titularidad.	Según la regla.	OK (7/7, 23/10/2026)

Concurrencia (conexiones simultáneas reales)

#	Escenario	Resultado esperado
1	Dos reservas simultáneas del mismo horario.	Una sola reserva.
2	Reservas simultáneas pegadas (margen de 15 min).	La segunda se rechaza.
3	Ráfaga de 5 reservas del mismo horario.	Una sola reserva.
4	Ráfaga de 3 reservas en una cochera de capacidad 2.	Dos reservas.
5	Mismo vehículo a la vez en dos cocheras.	Una sola reserva.
6	Dos reservas que juntas superan el saldo.	Una sola; saldo nunca negativo.

#	Escenario	Resultado esperado
7	Dos cancelaciones simultáneas.	Un solo reembolso.
8	Conductor y propietario cancelan a la vez.	Un solo reembolso.
9	Dos finalizaciones simultáneas.	Un solo cobro al propietario.
10	Cancelar mientras se finaliza.	Gana la cancelación; sin cobro.
11	Finalizar mientras se cancela.	Gana la finalización; sin reembolso.
12	Dos recargas simultáneas contra el tope.	El saldo no supera el tope.

Resultado obtenido: 12/12 OK en la corrida del 23/10/2026 sobre la versión final. En cada carrera la segunda operación esperó el bloqueo y falló sin duplicar dinero.

10.3 Pruebas funcionales realizadas

Flujo	Pasos	Resultado esperado	Obtenido
Registro y login	Registrarse, verificar, iniciar sesión con correo y con Google.	Sesión en el rol correcto.	OK (dev build Android)
Ciclo del conductor	Buscar → reservar → pagar → iniciar → finalizar → calificar.	Saldo debitado, propietario cobra, calificación visible.	OK
Cancelaciones	Cancelar con más y con menos de 1 h; cancelar como propietario.	Reembolso o rechazo según la regla.	OK
Publicación y moderación	Publicar, aprobar desde el panel, pedir cambios, reenviar, rechazar.	Estados y notificaciones correctos.	OK
Verificación de titularidad	Publicar con DNI y documentos, revisar en el panel y aprobar; comprobar que los documentos se borran.	Publicación aprobada con sello y sin documentos guardados.	OK (16/10/2026)
Soporte	Abrir ticket, responder desde el panel, resolver, reabrir.	Notificaciones y reapertura.	OK
Cierre automático	Dejar una reserva sin finalizar.	Se cierra sola 15 min después del fin.	OK
Panel	Dashboard, usuarios, reseñas, transacciones.	Datos reales.	OK
Dashboard y detalle del propietario, tickets por rol, capacidad, teclado en chats	Uso en la app.	Según la funcionalidad.	**OK (16/10/2026)**
Recuperación con código	Correo → código → contraseña.	Contraseña nueva.	Bloqueado hasta configurar SMTP propio
Notificaciones locales	Reserva para mañana y reserva por terminar.	Aviso en el celular.	OK con el build nuevo (16/10/2026)

10.4 Errores detectados y forma de resolución

Error	Causa	Resolución
El propietario nunca cobraba.	La migración 0015 reescribió `complete_booking` y eliminó el crédito al propietario.	`_finalize_booking` vuelve a acreditar (solo el precio pactado) — 0024.
Se cobraban 3 horas de excedente en cada reserva.	Se mezclaban horas sin zona con la hora del servidor (UTC).	Conversión explícita a hora argentina en todas las funciones y `shared/time.ts`.
«column does not exist» al finalizar.	Columnas creadas a mano desde el editor de tablas con nombres mal escritos.	Migraciones correctivas 0026–0027; regla: todo cambio por migración.
El cierre automático nunca cerraba nada.	Un excedente mayor al saldo violaba `balance >= 0` y abortaba todo el bucle.	Tope de 2 h y cobro de lo disponible — 0029.
Consultas fallaban con recursión infinita.	La política de cocheras consultaba reservas y viceversa.	Funciones auxiliares fuera de RLS — 0040.
Cualquiera podía volverse administrador o auto-aprobarse.	Una política no puede restringir columnas.	Disparadores de protección — 0033 y 0037.
Hallazgos de la auditoría (reservas manipulables, dobles reembolsos, calificaciones ajenas).	Escritura directa y falta de *locks*.	Migraciones 0046–0061 (ver 9.6).
El login con Google volvía a «localhost».	Supabase rechaza URLs de retorno con IP numérica (Expo Go).	*Development build* con esquema `garagego://`.
La app se rompía con React duplicado.	Metro leía las dependencias del panel.	`metro.config.js` excluye `admin-panel/node_modules`.
Pantallas que arrastraban datos de la visita anterior.	Pantallas ocultas en las pestañas en lugar de pilas anidadas.	Patrón de carpeta con `_layout` + `index` por sección.
Datos de otro usuario no aparecían sin error.	RLS filtra en silencio los cruces entre dueños.	Políticas acotadas a la relación real (0018).
Advertencia de botones anidados en web.	`Card` dibujaba un `<button>` dentro de otro.	Commit `f3089c2`.
Recargar una ruta en la web daba 404 / bucle infinito.	Falta de configuración SPA en Cloudflare.	`not_found_handling: single-page-application` en `wrangler.jsonc`.
El teclado tapaba el campo del chat.	Comportamiento distinto en Android e iOS.	`react-native-keyboard-controller`.
La prueba de recargas simultáneas (escenario 12) fallaba.	El escenario usaba el tope de saldo anterior ($ 50.000); la 0074 lo subió a $ 200.000, así que las dos recargas entraban de forma válida.	Se actualizó el escenario al tope vigente (saldo inicial $ 190.000): entra una recarga y la otra se rechaza (topup_limit_reached).

10.5 Limitaciones y funcionalidades que todavía requieren pruebas

- No hay pruebas automatizadas de interfaz ni de punta a punta (por ejemplo, Maestro o Detox para móvil, Playwright para web).
- Los scripts SQL y de concurrencia se ejecutan a mano contra la base; no forman parte del CI.
- iOS no fue probado en dispositivo (la clave de Maps de iOS sigue como marcador).
- Recuperación de contraseña de punta a punta (pendiente del SMTP propio).
- Migraciones 0070–0083: corridas en Supabase por el equipo. Verificadas con `ownership_checks.sql` (15/15, 16/10/2026), `cleanup_checks.sql` (8/8) y `photo_address_checks.sql` (7/7), los dos últimos el 23/10/2026.
- Accesibilidad con lector de pantalla, pruebas de carga y pruebas con usuarios reales.

11. Instalación y despliegue

11.1 Requisitos previos

- **Node.js 22.13 o superior** y npm (el SDK 57 lo exige; el CI usa Node 22).
- Git y acceso al repositorio.
- Un proyecto de **Supabase** con las extensiones `postgis` y `pg_cron` habilitadas.
- Una **clave de Google Maps**: Maps SDK for Android y Maps JavaScript API (para la web).
- Para Android: cuenta de **Expo / EAS** y un *development build* instalado (no Expo Go).
- Opcional: cuenta de **Cloudflare** para publicar la web y el panel.

11.2 Instrucciones para instalar y ejecutar

Paso	Comando / acción
1. Clonar	`git clone https://github.com/porcelcabralnicolas-27/GARAGEGO-2027-CLAUDE.git`
2. Instalar la app	`npm install` (en la raíz)
3. Configurar	Crear `.env` (ver 11.3) y la clave de Android en `app.json`
4. Ejecutar en Android	`npx expo start --dev-client` y abrir el *development build*
5. Ejecutar en la web	`npx expo start --web`
6. Panel	`cd admin-panel`, `npm install`, crear `.env`, `npm run dev` → http://localhost:5180
7. Verificar	`npx tsc --noEmit`, `npm test`, `npm run test:tz`
8. Generar APK	`eas build --platform android --profile preview`

11.3 Variables de entorno necesarias

Variable	Dónde	Descripción
`EXPO_PUBLIC_SUPABASE_URL`	`.env` raíz y variables de EAS	URL del proyecto de Supabase.
`EXPO_PUBLIC_SUPABASE_ANON_KEY`	`.env` raíz y variables de EAS	Clave pública (anon). **Nunca** la `service_role`.
`EXPO_PUBLIC_GOOGLE_MAPS_WEB_API_KEY`	`.env` raíz	Clave de Maps JavaScript API, restringida por dominio.
`android.config.googleMaps.apiKey`	`app.json`	Clave de Maps SDK for Android (restringida por paquete y firma).
`VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY`	`admin-panel/.env`	Mismos valores que la app.
`DATABASE_URL`, `GG_DRIVER_ID`, `GG_OWNER_ID`	Entorno de la terminal (solo pruebas)	Conexión *Session pooler* y dos usuarios de prueba para `test:concurrency`.

11.4 Configuración de la base de datos

1. Crear el proyecto en Supabase y habilitar las extensiones **postgis** y **pg_cron** (Database → Extensions).
2. Ejecutar **en orden** las 83 migraciones de `supabase/migrations/` desde el SQL Editor (o `supabase db push` con la CLI). Todas son idempotentes. Cuidado con el orden indicado en sus encabezados: la 0051 va después de publicar la app que lee `public_profiles`; la 0061 después de la 0059.
3. Authentication → Providers: habilitar **Email** (con longitud del código de recuperación) y **Google** (client IDs de Google Cloud).
4. Authentication → URL Configuration: *Site URL* y *Redirect URLs* con `garagego://**` y la URL web publicada.
5. Authentication → Emails: configurar **SMTP propio** y la plantilla «Reset Password» con `{{ .Token }}`; usar `docs/plantilla-mail-confirmar-cuenta.html` para la confirmación.
6. Los *buckets* de Storage, las publicaciones de Realtime y los jobs de pg_cron los crean las migraciones.
7. Crear un administrador: registrar la cuenta y ejecutar en el SQL Editor `update public.profiles set is_admin = true where id = '<id>';`.
8. Verificar con los scripts de `supabase/tests/` (completando los ids de prueba de cada encabezado).

11.5 Datos y usuarios iniciales para pruebas

El repositorio **no incluye un script de datos iniciales** (*seed*). Para preparar una demostración:

1. Registrar una cuenta de conductor y otra de propietario.
2. Marcar una tercera cuenta como administradora (paso 7 anterior).
3. Con el propietario, publicar 3 a 5 cocheras con datos realistas y aprobarlas desde el panel.
4. Con el conductor, cargar un vehículo y saldo de prueba (botón de recarga, hasta $200.000).

Cuenta	Rol	Correo	Contraseña
Demo conductor	Conductor	garagego2027+conductor@gmail.com	Entregar por un canal privado
Demo propietario	Propietario	garagego2027+propietario@gmail.com	Entregar por un canal privado
Administrador	Panel	Solo para uso del equipo de GarageGo	No se entrega

Los dos correos de demostración son alias de la casilla del proyecto (garagego2027@gmail.com): Gmail entrega en esa misma bandeja todo lo que va después del «+», y para el sistema son dos cuentas distintas. Hacen falta dos porque nadie puede reservar

su propia cochera. El panel de administración es de uso exclusivo del equipo de GarageGo; si hace falta mostrarlo, se presenta en vivo.

11.6 Procedimiento de despliegue

Componente	Procedimiento
App web	`npm run build:web` y `npx wrangler deploy` en la raíz (Worker `garagego`). Agregar el dominio a la clave de Maps y a las Redirect URLs de Supabase.
Panel	`cd admin-panel`, `npm run build` y `npx wrangler deploy` (Worker `garagego-admin`).
App Android	`eas build --platform android --profile preview` → APK de distribución interna. EAS empaqueta **desde Git**: commitear antes; las variables se cargan en EAS.
Base de datos	Las migraciones nuevas se ejecutan a mano en el SQL Editor; no requieren redesplegar los clientes salvo que cambie la API.
CI	Automático en cada push (GitHub Actions).

11.7 Dirección de acceso

Recurso	Dirección
App web	https://garagego.garagego2027.workers.dev
Panel de administración	https://garagego-admin.garagego2027.workers.dev
APK de Android	https://expo.dev/artifacts/eas/eC6hujrfHuFduDvULk357acyMvGc_111Z11kradBdaE.apk
Repositorio	https://github.com/porcelcabralnicolas-27/GARAGEGO-2027-CLAUDE

12. Repositorio y control de versiones

12.1 Enlace al repositorio

https://github.com/porcelcabralnicolas-27/GARAGEGO-2027-CLAUDE

12.2 Archivo README

El `README.md` de la raíz explica el problema, el funcionamiento por rol, las tecnologías y su motivo, los requisitos, la instalación, la ejecución, la estructura de carpetas y el índice de la documentación, los integrantes y el enlace al diseño en Figma.

12.3 Organización de carpetas y componentes

Ruta	Contenido
`app/`	Pantallas y rutas (expo-router): `(auth)`, `(client)`, `(owner)`.
`src/features/`	Lógica por dominio: `api.ts`, `types.ts`, `schemas.ts`, pantallas y tests.
`src/components/`	Componentes reutilizables (`ui/`, `booking/`, `parking/`).
`src/lib/`, `src/store/`, `src/theme/`, `src/hooks/`	Cliente de Supabase y utilidades, estado global, tokens de diseño, hooks.
`shared/`	Dominio y horas compartidos con el panel.
`supabase/migrations/`	83 migraciones SQL: esquema, RLS, funciones, disparadores y jobs.
`supabase/tests/`	9 scripts de verificación SQL.
`scripts/`	Pruebas de concurrencia y de zona horaria.
`admin-panel/`	Panel de administración (proyecto Vite independiente).
`docs/`	Requerimientos, historias, reglas, casos de uso, arquitectura, base de datos, seguridad, UI/UX y esta documentación.
`.github/workflows/ci.yml`	Integración continua.
`app.json`, `eas.json`, `wrangler.jsonc`, `metro.config.js`	Configuración de Expo, builds, Cloudflare y empaquetador.

12.4 Instrucciones de ejecución

Script	Qué hace
`npm start` / `npm run android` / `npm run web`	Servidor de desarrollo de Expo.
`npm run build:web`	Exporta la versión web a `dist/`.
`npm test` / `npm run test:watch`	Tests unitarios.
`npm run test:tz`	Tests de fechas en 6 zonas horarias.
`npm run test:concurrency`	Pruebas de concurrencia contra la base (requiere variables).

Script	Qué hace
`admin-panel`: `npm run dev` / `build` / `typecheck`	Panel en desarrollo, build y verificación de tipos.

12.5 Historial de commits

61 commits con mensajes descriptivos en español con la forma «Área: cambio» (por ejemplo, «Reservas: un vehículo por reserva a la vez y recordatorio un día antes»). Se trabajó con **una rama por etapa**:

Rama	Contenido
`master`	Commit inicial y alineación con el SDK 57 (3 commits).
`rediseno/figma`	Rediseño visual y clave de Maps.
`feature/pagos`	Flujo de pago, sesión activa, historial y notificaciones por rol.
`feature/panel-admin`	Panel de administración.
`feature/auditoria-seguridad`	Correcciones de la auditoría y backlog técnico.
`feature/ultimas-funcionalidades`	Funcionalidades finales (rama actual).
`feature/arreglos-perfil`, `mapa/nuevo`	Ramas de trabajo puntuales.

12.6 Versión presentada

Versión	1.0.0 (`package.json` y `app.json`)
Rama	`feature/ultimas-funcionalidades`
Último commit	361d411 (16/10/2026), con las migraciones hasta la 0082; la 0083 se suma en el próximo commit
Etiqueta sugerida	v1.0.0 sobre el commit final una vez unida la rama principal: 361d411, o el commit que sume docs/ y el README si se commitean antes de entregar

13. Estado final y conclusiones

13.1 Funcionalidades terminadas

- Ciclo completo del conductor: búsqueda (ubicación o dirección, mapa con disponibilidad ahora, lista y filtros), ficha, ruta, reserva con vehículo, pago con saldo, comprobante, inicio, finalización, cancelación, historial y calificación.
- Ciclo completo del propietario: asistente de publicación con verificación de titularidad, tipo, capacidad, servicios, disponibilidad y días bloqueados, fotos, moderación con cambios validados, reservas recibidas, cobro automático, dashboard y estadísticas.
- Chat en tiempo real, notificaciones por rol, recordatorios, centro de ayuda con tickets.
- Cierre automático, excedente con tolerancia y tope, reglas de concurrencia probadas.
- Panel de administración completo y conectado a datos reales.
- Seguridad corregida según la auditoría, CI, tests y despliegue web.

13.2 Funcionalidades parcialmente desarrolladas

Funcionalidad	Estado
Pagos	Billetera simulada completa; Mercado Pago diseñado sin implementar.
Notificaciones	Dentro de la app y locales; sin push remoto.
Recuperación de contraseña	Implementada; falta el SMTP propio para usar el código en producción.
Eliminación de cuenta	Botón visible sin funcionalidad.
Tarjetas guardadas, retiro de fondos	Visibles como «Próximamente».
Modo oscuro	Paleta definida, sin activar.
iOS	Compilable, no probado.

13.3 Problemas conocidos

- Dependencia del servidor demo de OSRM (sin garantía).
- Sesión sin cifrar en el dispositivo.
- El excedente que el conductor no puede pagar se pierde (consecuencia del saldo simulado).
- Textos legales en borrador.

13.4 Decisiones que debieron modificarse

Decisión original	Cambio	Motivo
El propietario aprueba cada reserva.	Confirmación inmediata al pagar; el propietario puede cancelar.	Menos espera e incertidumbre para el conductor.

Decisión original	Cambio	Motivo
Google Directions vía Edge Function.	OSRM desde el cliente.	Costo cero.
Probar con Expo Go.	*Development build* de EAS.	El login con Google no funciona en Expo Go.
Paleta amarilla.	Verde lima de Figma con tokens de contraste.	Rediseño y accesibilidad AA.
Capacidad 1 con restricción de exclusión.	Capacidad 1–50 con *locks* por cochera.	Estacionamientos con varios lugares.
Chat por reserva.	Un hilo por cochera.	El historial no se fragmentaba en cada reserva.
Cierre automático a las 2 h, job cada 15 min.	Cierre a los 15 min, job cada minuto, avisos previos.	Liberar antes el lugar y avisar al conductor.
Borrar una cochera en cualquier momento.	Bloquear o archivar según las reservas.	Se perdían reservas pagadas e historial.
Escritura directa sobre reservas.	Solo funciones del servidor.	Hallazgo crítico de la auditoría.
Recuperación con enlace.	Recuperación con código.	Más simple en el celular y permite a cuentas Google crear contraseña.
Tickets siempre vinculados.	Consultas generales permitidas.	Había reclamos que no eran de una reserva o cochera.
Panel con datos simulados.	Panel conectado a la base.	El diseño se validó primero con mocks.
Editar una publicación aprobada no la revisaba.	Vuelve a revisión si cambian datos, tarifa, servicios o fotos.	Que lo publicado siga siendo lo que el equipo aprobó.
Publicar sin acreditar la titularidad.	DNI y documentos obligatorios, borrados al decidir.	Evitar publicaciones de lugares ajenos y guardar datos personales solo el tiempo necesario.

13.5 Principales dificultades encontradas

Dificultad	Cómo se enfrentó
Mezcla de horas sin zona con la hora del servidor.	Regla única: todo en hora de Buenos Aires, centralizado y probado en 6 husos.
Base con migraciones aplicadas a medias o columnas creadas a mano.	Migraciones correctivas y la regla de que todo cambio de esquema entra por una migración idempotente.
Carreras y dobles cobros invisibles en pruebas manuales.	Auditoría externa, *locks*, índice único contable y pruebas con conexiones simultáneas.
Agujeros de seguridad por políticas que no restringen columnas.	Disparadores de protección y revisión sistemática.
Login con Google en el celular.	Diagnóstico de la causa (IP numérica) y paso al *development build*.
Convivencia de dos proyectos en un repositorio.	`metro.config.js` y `tsconfig` con exclusiones.

Dificultad	Cómo se enfrentó
Costo cero.	OSRM, billetera simulada y hosting gratuito.

13.6 Logros del equipo

- Un producto completo y funcional en tres plataformas y un panel de administración, con un único backend y sin servidores propios.
- Reglas de negocio garantizadas por la base: ningún cliente modificado puede reservar un horario ocupado, duplicar un reembolso ni asignarse permisos.
- Corrección integral de una auditoría técnica, con verificación automatizada y contra la base real.
- Documentación extensa dentro del código (cada migración explica el problema y la decisión) y en `docs/`.
- Versión web pública para mostrar el sistema.

13.7 Aprendizajes obtenidos

- **La seguridad no puede depender del cliente:** todo lo que viaja al usuario es público.
- **Elegir la base por el modelo de datos:** el problema de reservas es relacional y transaccional.
- **La concurrencia se prueba, no se supone:** los errores de dinero aparecieron recién con dos operaciones simultáneas.
- **El tiempo es un dato del dominio:** fijar una zona horaria evitó una familia entera de errores.
- **Documentar las decisiones, no solo el código:** las decisiones Q1–Q14 y los encabezados de las migraciones evitaron reabrir debates.
- **Una auditoría externa encuentra lo que el equipo no ve.**
- **Accesibilidad desde el sistema de diseño:** una sola decisión de tokens corrigió el contraste en toda la app.

13.8 Mejoras y próximos pasos

Prioridad	Próximo paso
Inmediata	Activar Cloudflare Access en el panel y configurar SMTP propio.
Alta	Integrar Mercado Pago según el diseño cerrado (Edge Functions, webhook, reconciliación).
Alta	Eliminación de cuenta y política de datos personales.
Media	Sesión en almacenamiento seguro, notificaciones push remotas, tests end-to-end, observabilidad.
Media	Servidor de rutas propio o proveedor con garantía.
Baja	Modo oscuro, auditoría con lector de pantalla, publicación en tiendas, retiro de fondos.

Anexo A. Sugerencias: qué falta para cumplir la consigna

Revisión de cada punto de la consigna contra lo que hoy existe en el proyecto.

Punto de la consigna	Estado	Acción sugerida
1 · Integrantes y roles	Hecho	Nombres y roles en la tabla 1.2, en la portada y en el README.
4 · Backlog y registro de tareas	Reconstruido	Si el equipo usó Trello, Jira, GitHub Projects o similar, adjuntar capturas o el enlace como evidencia; si no, crear un tablero con los ítems de 4.3.
8 y 12 · Capturas con datos realistas	Parcial	Las capturas de la app y del panel están en la sección 8.1; falta insertarlas en el manual de usuario.
8 · Diseño en Figma	Hecho	Enlace en el README: figma.com/design/oGHdhF6kc1ydCaQjtDKK6t/GARAGEGO.
10 · Resultados obtenidos	Hecho	Scripts SQL de las últimas migraciones y test:concurrency (12/12) corridos sobre la versión final el 23/10/2026.
10 · Pruebas de interfaz	Falta	Al menos un recorrido end-to-end automatizado o un registro de pruebas manuales con fecha, responsable y resultado.
11 · Datos iniciales	Hecho	Cuentas de demostración de conductor y propietario en la sección 11.5; el panel queda para uso del equipo.
11 · Dirección del panel y del APK	Parcial	El panel está publicado en garagego-admin.garagego2027.workers.dev; falta el enlace del APK (11.7).
13 · Historial de commits del equipo	Explicado	Un único autor porque Bruno Bellot integró el trabajo de cada integrante.
13 · Versión presentada	Falta	Crear la etiqueta `v1.0.0` y anotar el hash.
13 · README actualizado	Hecho (sin commitear)	README actualizado con 83 migraciones, integrantes y Figma; falta commitearlo.
9 · Seguridad	Mitigado	La clave de Android ya está restringida por paquete y huella SHA-1; sigue en app.json porque viaja dentro del APK de todos modos.
Presentación oral	Sugerencia	Preparar una demo en vivo con la versión web y el panel, y un plan B con video por si falla la red.

Anexo B. Glosario

Término	Significado
Cochera / estacionamiento	Espacio publicado; puede ser de una casa (`house`) o un predio con varios lugares (`lot`).
Conductor / propietario	Los dos roles de una misma cuenta.
Margen (buffer)	15 minutos antes y después de cada reserva para que un auto salga antes de que entre el siguiente.
Excedente (overtime)	Tiempo usado después del fin reservado; se cobra a 1,5 × el precio por hora pasada la tolerancia.
No-show	Reserva a la que el conductor nunca llegó.
RLS (Row Level Security)	Políticas de PostgreSQL que deciden qué filas puede ver o modificar cada usuario.
RPC	Llamada a una función de la base a través de la API.
`security definer`	Función que corre con los permisos de su dueño y valida internamente quién la llama.
Advisory lock	Bloqueo lógico de PostgreSQL usado para serializar reservas de una misma cochera o vehículo.
Migración	Archivo SQL numerado que aplica un cambio al esquema de forma reproducible.
Development build	Versión de la app compilada con EAS que incluye las herramientas de desarrollo y el esquema propio.
Anon key / service_role	Clave pública del proyecto / clave privilegiada que ignora RLS (nunca en el cliente).
Titularidad	Acreditación de que quien publica es dueño del lugar o tiene permiso para ofrecerlo.
URL firmada	Enlace temporal a un archivo privado de Storage, que vence a los pocos minutos.

Anexo C. Migraciones por etapa

Rango	Qué introdujo
0001–0007	Esquema inicial, ingreso con Google, perfil, imágenes y búsqueda.
0008–0014	Reservas, billetera, cobro al propietario, calificaciones, horarios ocupados, chat y notificaciones en tiempo real.
0015–0023	Margen y excedente, vehículos, inicio de reserva, notificaciones de mensajes, reglas de horario, acceso al chat, límites.
0024–0032	Flujo de pago y sesión activa, migración de reservas viejas, correcciones de columnas, cron de cierre, tope de excedente, notificaciones por rol, historial, búsqueda sin cocheras propias.
0033–0045	Administración: marca de administrador, moderación, soporte, dashboard, usuarios, reseñas y transacciones.
0046–0053	Auditoría: bloqueo de escritura de reservas, máquina de estados, reglas en el servidor, integridad de calificaciones, privacidad de perfiles, privilegios de funciones, archivado de cocheras.
0054–0061	Backlog: tickets atómicos, vistas previas de mensajes, directorio agrupado, tope de recarga, límites de Storage, listados paginados, Realtime del panel, recarga demo segregada.
0062–0069	Detalle de reserva del propietario, tickets por rol, capacidad, servicios, cierre a los 15 min, un vehículo por reserva, recordatorio 24 h, vehículo en el detalle.
0070–0076	Remitente en notificaciones de mensaje, limpieza de excepciones vencidas, cambios solicitados validados, disponible ahora, topes de recarga mayores, tipo de publicación, consultas generales de soporte.
0077–0082	Fotos de vehículos privadas, verificación de titularidad, borrado de documentos al decidir, cierre de chats al eliminar una publicación, re-revisión al editar, foto obligatoria y nueva titularidad al cambiar la dirección.
0083	Tope de búsqueda de 1000 km: el mapa de Inicio muestra todos los estacionamientos publicados y la distancia solo se limita desde «Buscar».